Prepared for submission to JHEP

dSIBP 用户手册

Abstract: 本手册说明dS IBP package 的费曼规则、massive/massless 指标约定、time 与loop-momentum IBP、独立变量微分方程seed、EOM/缩并canonical、拓扑输入、公开API、Kira 接口和tree vertex family。公开topology 只使用严格的vertices/lines Association，顶点以vertexType 和externalLegEnergy 定义外腿指数；full 模式使用J[aList,linePacks,ispList]，time-only 模式使用J[sectorKey,timeShifts,stateBits]。用户显式分别声明任意命名的loopExternalMomenta 与independentExternalMomenta，程序验证结构圈数、routing、声明完备性和动力学坐标，不自动猜角色。package 只生成、序列化和验证关系，不运行reduction。

Contents

1 快速开始与当前边界 2

2 完整例子：mix bubble+tree 4

3 SK 费曼规则到积分族 8

4 按mode 固定的三参数积分表示 9

5 Massive 导数、EOM 与缩并 10

6 Massless 双顶点quotient 与time-IBP 12

7 Topology、标量积、ISP 与外腿能量 15

8 IBP 生成元与公开接口状态 20

9 Seed、linearData 与serializer 21

10 标准package 与初始化 27

11 运行消息与进度 28

12 Kira 结果取回到微分方程 28

13 Tree vertex family 31

14 常见错误与当前限制 33

A 公开函数汇总 33

dSIBP Handbook 36

1 Scope and quick start 36

2 A complete mixed bubble-tree example 37

3 From SK rules to integral families 37

4 Integral representations 38

5 Massive function systems, derivatives, and EOM 38

6 Massless quotients and time IBP 39

7 Topology, scalar products, ISP, and symmetry **39**

8 IBP generators and atomic operations **39**

9 Seeds and continuous-index envelopes **40**

10 Linear data and the Kira workflow **41**

11 Initialization, messages, and inspection **41**

12 Pure massive bubble closed loop **42**

13 Tree vertex families **42**

14 Common errors and current boundaries **42**

15 Public API summary **43**

---

1 快速开始与当前边界

本package 生成dS 圈图的canonical IBP seed，并把关系转换为后端中立的`linearData`。它支持topology-driven 的任意圈数、任意拓扑以及massive/massless 混合内部传播子；唯一加载入口是发布目录内的标准模块树：

```
dSIBP/
  dSIBP.m
  Kernel/
```

本package 使用严格的干净输入schema，并保留标准context、初始化metadata、完整time/loop-momentum 生成元、massive EOM、共同-theta odd-subset contact、contact 可达sector、跨sector coincident canonical、Kira serializer/importer 以及DE/scaling 检查。full 模式保留双顶点三槽传播子pack；time-only 使用显式sector key、compact 时间幂和building-block 状态位列表。

在新kernel 中把发布目录加入`$Path` 后加载：

```
packageDir = FileNameJoin[{Directory[],"dSIBP"}];
If[!MemberQ[$Path,packageDir],AppendTo[$Path,packageDir]];
Needs["dSIBP`"];
```

该入口提供`dSIBP`` 公开context。`Examples/` 内的每个示例都通过`Examples/load_current_package.wl` 使用同一个入口，因此不需要调用方设置路径。

全部模块成功加载后，每个Wolfram kernel 只显示一次简洁引用提醒；Notebook 中的arXiv 号是可点击链接，headless kernel 同时保留完整URL。使用dSIBP 开展研究时，建议引用：

1. Jiaqi Chen and Bo Feng, "Towards Systematic Evaluation of de Sitter Correlators via Generalized Integration-By-Parts Relations", arXiv:2401.00129.
2. Jiaqi Chen, Bo Feng and Yi-Xiao Tao, "Multivariate hypergeometric solutions of cosmological (dS) correlators by d log-form differential equations", arXiv:2411.03088.
3. Jiaqi Chen, Bo Feng, Zhehan Qin and Yi-Xiao Tao, "Loop integrals in de Sitter spacetime: The parity-split IBP system and d log-form differential equations", arXiv:2604.14549.
4. dSIBP package paper, arXiv identifier pending.

程序包论文尚未提供arXiv 号，因此本手册不为它生成链接或书目信息。

推荐工作流为

1. 输入顶点、内部传播子、圈/外动量基、ISP、零点和必要的数值规则；
2. 用`DSInit` 验证输入、ISP 坐标、函数系统与contact-reachable sectors；
3. 用`DSSeeds` 生成普通canonical seed batch，并从稳定字段`"allSeeds"` 或`DSAllSeeds` 取得全部sector/generator 的一维general 模板；massive 完整遍历顶点四态，masslessFull 只生成$n_2 = 0$ 的00/10 quotient representatives；

4. 模板层立即执行EOM/symmetry/canonical；再用唯一入口`DSGenerateIBP` 显式展开连续指标；

5. 用`DSLinear` 把撒点后的canonical batch 构造成后端中立`linearData`；必要的小规模数值规则只在这一层代入；

6. 按需调用`DSKiraExport`；用户在package 外运行reduction；

7. 有完整外部结果时用`DSKiraImport -> DSDE -> DSScaleCheck` 取回并验证。

> 解析seed 只允许小规模生成。禁止默认展开整族解析IBP，也禁止为了验证做宽范围撒点。所有WolframScript 检查必须显示消息运行，不能用`Quiet` 掩盖问题。

给定下文的`case` 后，最短的可执行调用链是

```
context = DSInit[case];
templateData = DSSeeds[context];
allSeeds = DSAllSeeds[templateData];
seeds = DSGenerateIBP[allSeeds,{0,0}];
linearData = DSLinear[seeds,context];
{context["status"],templateData["dSIBPStatus"],
 seeds["dSIBPStatus"],
 linearData["dSIBPStatus"]}
```

每个批量接口都返回Association；必须先检查`"status"`，再读取方程或传给下一阶段。

发布目录`Examples/` 提供六个应用例子：

```
01_mixed_bubble_workflow.wl
02_function_system_hankel.wl
03_single_massive_sunrise/main.wl
04_pure_massive_bubble_closed_loop/main.wl
05_tree_two_vertex_time_ibp/main.wl
06_mix_bubble_tree/main.wl
```

建议先运行`06_mix_bubble_tree/main.wl`；它展示显式动量角色、`ssij/sEe` 初始化、参数重定义、总导数和cycle/fixed 传播子数据。两条cycle propagator 分别取`b[e]+n[e,1]+n[e,2]==0 (mod 2)` 的全偶子空间；fixed bridge 不属于圈积分，不进入parity constraints。`03_single_massive_sunrise/main.wl` 使用共同外腿能量`kE`、圈外动量`kL`、两个在massless 传播子交换下互换的ISP，以及顶点/平行传播子`symmetryRules`；它只生成全部contact-reachable sector 的general IBP seeds 和`{ss11,kE}` general 参数微分算符。连续指标保持符号，不进入连续撒点、`linearData`、Kira、DE 或scaling。

`01_mixed_bubble_workflow.wl` 只在内存中构造pre-reduction plan；03 与05 不运行Kira，05 也不写初始化目录。`04_pure_massive_bubble_closed_loop/main.wl` 展示外部Kira 回读、DE 与scaling；`06_mix_bubble_tree/` 同时给出其81-master 完整流程的轻量输入/结果摘要。examples 不含独立expected，也不启动reduction。真实输入工作树由Windows Wolfram 写入仓库外目录，Kira 2.3 在WSL 的对应目录运行；发布

副本只保留可读输入与摘要，已删除`init/`、`kira/`、database、save、日志、reduction table、cache 和DE 运行目录。

关于AI 使用的说明

本package 的notation、功能与接口由作者设计；作者指导多种agent 软件与大语言模型完成代码与文档的实现，并设计验证任务以确认所开发程序包的可靠性。验证任务均为计算结果的交叉验证，逐项说明其覆盖对象（IBP 关系、微分方程DE、scaling 关系）与数据来源：

- **massive 两顶点函数族**（IBP 与DE）：general IBP seeds 与DE 与文献arXiv:2411.03088 的解析结果比较；该族有5 个normalized master，三张5×5 DE 矩阵逐元exact 相等（每张25/25）。vertex basis 与master 排序取自文献arXiv:2401.00129。
- **massive 三顶点树图**（tree IBP 与tree DE）：公式先由agent 在此前手写两顶点代码的基础上独立推广推导，与文献arXiv:2604.14549 对上后再与程序包结果比较。
- **单圈bubble 全流程**（IBP、DE、scaling）：外部Kira 约化闭合（无unreduced target），返回19 个master；两张19×19 DE 矩阵对`{ss11,P0}` 保持符号，在记录的精确点上与文献arXiv:2604.14549 的矩阵逐元相等（三种变量约定各361/361）；scaling 恒等式符号成立（source-level 33/33、active basis 19/19）。
- **mix bubble+tree 全流程**（IBP、DE、scaling）：81 个master 的外部Kira 约化闭合；六张81×81 DE 无残留符号；81/81 个source-level Euler scaling 与IBP 恒等式严格为零，提供独立于数值约化的scaling 证据；约化自身的scaling 证书在记录的精确点给出。
- **通用公式对naive 撒点**（IBP 与DE）：程序包直接代入推导所得的通用公式，其输出与同一函数族naive IBP 种子撒点的约化结果比较一致。
- **树图DE 对MadStree**（tree DE）：直接$d\log$ 路线与naive IBP 路线结果一致；另对15 个master 的mixed 三顶点树图族，把`dSIBP` 的参数导数送入MadStree（MadStree package paper, arXiv identifier pending）对$k_1, k_2, k_3, \mathtt{sE1}, \mathtt{sE2}$ 五个变量约化，75 个导数全部闭合，五张15×15 矩阵与MadStree 直接$d\log$ 公式路线逐元相等。

程序包发布版见`https://github.com/J7C/dSIBP-Packages`。

2 完整例子：mix bubble+tree

2.1 先按独立相位能量走通基准流程

考虑三个时间顶点。`v1,v2` 之间的bubble 有两条cycle 传播子，动量分别为`l1` 与`l1+k1+k2`；`v2,v3` 之间有一条无圈bridge 传播子，动量为`k1+k2`。`v1` 的另一侧外腿也携带`k1+k2`，`v3` 有两条外腿`k1,k2`。输入中的加减法始终是矢量线性组合：

$$|k_1 + k_2| = \sqrt{\mathrm{sp}(k_1 + k_2, k_1 + k_2)} \neq |k_1| + |k_2| \tag{2.1}$$

（一般运动学下）。传播子1 是唯一massive h cycle 传播子，`nu1` 是它的Hankel 指标；传播子2 是massless exponential cycle 传播子，传播子3 是massless exponential bridge。`E1,E2,E3` 是三个顶点指数相位的独立标量。package 不从传播子动量或`nu1` 推断相位变量。

完整核心输入为

```
case = <|
  "name" -> "mixBubbleTreeK1K2",
  "vertices" -> {
    <|"id"->v1,"vertexType"->"+","externalLegEnergy"->E1|>,
    <|"id"->v2,"vertexType"->"+","externalLegEnergy"->E2|>,
    <|"id"->v3,"vertexType"->"+","externalLegEnergy"->E3|>
  },
  "lines" -> {
    <|"id"->1,"endpoints"->{v1,v2},"momentum"->l1,
      "nu"->nu1,"massType"->"massive"|>,
    <|"id"->2,"endpoints"->{v1,v2},"momentum"->l1+k1+k2,
      "massType"->"massless"|>,
    <|"id"->3,"endpoints"->{v2,v3},"momentum"->k1+k2,
      "massType"->"massless"|>
  },
  "extLegs" -> {
    {bubbleLeg,v1,k1+k2},{treeLeg1,v3,k1},{treeLeg2,v3,k2}
  },
  "loopMomenta" -> {l1},
  "loopExternalMomenta" -> {k1+k2},
  "independentExternalMomenta" -> {k1,k2},
  "ibpMode" -> "full",
  "ispData" -> {},
  "zeroPointRules" -> {
    a0[v1]->alpha1,a0[v2]->alpha2,a0[v3]->alpha3,
    b0[1]->beta1,b0[2]->beta2,b0[3]->beta3
  },
  "symmetryRules" -> {},
  "parityConstraints" -> {
    b[1]+n[1,1]+n[1,2] -> 0,
    b[2]+n[2,1]+n[2,2] -> 0
  }
|>;
```

前两条约束分别选择massive cycle 与massless cycle 的全偶子空间。传播子3 的`"F"` pack 是fixed bridge，不属于圈积分，不能出现在parity constraints 中。

先查看提案，再初始化：

```
proposal = DSKinematics[case];
proposal["selectionTemplate"]

context = DSInit[
  case,
```

```
  GenerateDerivativeMetadata -> True,
  ProgressReporting -> True
];
topology = context["topology"];
notation = DSParameterNotation[context];
```

该输入的必要loop 方向只有`k1+k2`。它的完整一维Gram 已覆盖bridge 的模长；`k1,k2` 只补齐实际出现的两个独立无圈模长。缺省规则严格为

```
sp[k1+k2,k1+k2] -> ss11^2
sp[k1,k1]       -> sE1^2
sp[k2,k2]       -> sE2^2
```

因此`ss11` 是$|k_1+k_2|$，而`sE1+sE2` 才是$|k_1|+|k_2|$。短名必须与`DSParameterNotation` 返回的原始`sp` binding 一起阅读；`ssij` 在$i \neq j$ 时是Gram 点积坐标的根，不表示$|k_i+k_j|$。

初始化后可以检查公开原子关系和批量工作流:

```
integral = J[
  {0,0,0},
  {{0,0,0},{0,0,0},{"F",0,0}},
  {}
];

timeSeed = dtau[v3,integral,topology];
loopLoopSeed = dqq[1,1,integral,topology];
loopExternalSeed = dqk[1,1,integral,topology];
totalDerivative = ds[ss11^2 integral+sE1 integral,sE1,topology];

seeds = DSSeeds[context];
linearData = DSLinear[seeds,context];
```

massive/massless cycle full pack 都是`{b,n1,n2}`；fixed bridge full pack 是`{"F",n1,n2}`，其中`"F"` 是数据sentinel，不是幂指标。bridge 的动量幂进入sector 的结构化prefactor，不伪造`b` 指标。解析seed 不要求给数值规则。即使为了缩小Kira 系数域固定`dim`、`nu1`、zero-point 或normalization，所有准备生成DE 的变量`{ss11,sE1,sE2,E1,E2,E3}` 仍必须保持符号；不得在export 前给它们数值。

本例对应的formal Kira 输入与最终结果分别摘要在`06_mix_bubble_tree/kira_input_summary.wl` 和`06_mix_bubble_tree/kira_result_summary.wl`。fresh 输入含178 个seed templates、818217 条canonical relations、818297 条formal equations、310769 个积分、81 个active masters 和676 个formal targets。WSL Kira 2.3 以`w10*1` 在160.11 s 墙钟时间内完成，`unreduced=0`；六张DE 均为81×81。reduction 的scaling 证书只覆盖记录的exact point，全符号齐次性另由81/81 个source-level Euler+time/q-IBP identities 提供。摘要不作为`DSKiraImport` 输入；完整工作树已在发布前删除。

2.2 方案一：保留两条外腿，把模长和选作坐标

如果同一顶点的两个外腿指数需要合并成

$$E_0 = |k_1| + |k_2|, \tag{2.2}$$

是否采用E_0 是用户的运动学参数选择，不由package 猜测。保留原topology 时，先把v3的相位能量显式改为E0，再做满秩坐标重定义：

```
boundCase = Join[
  case,
  <|"vertices"->ReplacePart[
      case["vertices"],{3,"externalLegEnergy"}->E0
    ]|>
];
boundContext0 = DSInit[boundCase,GenerateDerivativeMetadata->True];

boundContext = DSRedefineParameters[boundContext0,{
  sp[k1+k2,k1+k2] -> ss11^2,
  sp[k1,k1]       -> (E0-sE2)^2,
  sp[k2,k2]       -> sE2^2
}];
```

数学上这正是sE1->E0-sE2,sE2->sE2，但公开API 的规则左端必须写baseCoordinateOrder 中的原始sp[...]，不能直接写sE1->...。该坐标图的可求导变量为

```
{ss11,E0,sE2,E1,E2}
```

其中E0 只出现一次；ds[expr,E0,boundContext] 同时对显式系数、模长binding 和v3 相位求导。平方规则选择了$|k_1| = E_0 - sE_2$ 的物理支，使用者应在所研究区域保证$E_0 > s_E2 > 0$；若需要别的分支，必须显式更换参数化。

只改坐标而仍令v3->E3，不会建立$E_3 = E_0$；相反，只令v3->E0 而不重定义两个模长，也不会告诉package $E_0 = sE1 + sE2$。这两部分必须同时给出。

2.3 方案二：从输入起采用单一有效外腿

另一种做法是把v3 的两条外腿替换成一条有效外腿。此时topology metadata 已改变，不能把它当作上一个family 的纯坐标改名。为避免同一符号既作矢量又作标量，使用p0 标记有效外腿三动量，用E0 标记其模长和顶点相位：

```
singleLegCase = Join[case,<|
  "name" -> "bubbleTreeSingleEffectiveLeg",
  "extLegs" -> {
    {bubbleLeg,v1,k1+k2},
    {treeLeg0,v3,p0}
  },
  "independentExternalMomenta" -> {p0},
  "vertices" -> ReplacePart[
    case["vertices"],{3,"externalLegEnergy"}->E0
  ]
|>];

singleLegContext0 = DSInit[
  singleLegCase,
```

```
  GenerateDerivativeMetadata->True
];
singleLegContext = DSRedefineParameters[singleLegContext0,{
  sp[k1+k2,k1+k2] -> ss11^2,
  sp[p0,p0]       -> E0^2
}];
```

此时v3 只有一条外腿，独立变量为{ss11,E0,E1,E2}。package 只知道$|p_0| = E_0$；“这个E_0 对应原问题的$|k_1| + |k_2|$”是用户在两个不同topology 之间建立的外部物理对应，不再能从新输入中恢复原来的两个独立模长，也不能分别对它们求导。若后续仍需要k_1, k_2 的独立变化、交换对称性或软极限，应使用方案一而不是删除两条外腿。

3 SK 费曼规则到积分族

3.1 顶点规则

每个时间顶点v 有共形时间$\tau_v < 0$ 和SK 分支$\sigma_v = \pm$。本package 使用

$$\sigma_v = +: \quad e^{-iE_v\tau_v}, \qquad \sigma_v = -: \quad e^{+iE_v\tau_v}. \tag{3.1}$$

因此外腿相位对time-IBP 的贡献为

$$\partial_{\tau_v} e^{-iE_v\tau_v} = -iE_v e^{-iE_v\tau_v}, \qquad \partial_{\tau_v} e^{+iE_v\tau_v} = +iE_v e^{+iE_v\tau_v}. \tag{3.2}$$

E_v 表示连接该顶点的所有外腿模长在指数中形成的打包能量。它不一定是进入含圈传播子动量的外部向量。

3.2 内部传播子的四种类型

设内部传播子e 的有序顶点为$\{u_e, v_e\}$，动量为

$$\boldsymbol{Q}_e = \sum_{\ell=1}^{L} c_{e\ell}\boldsymbol{q}_\ell + \sum_{j=1}^{K} d_{ej}\boldsymbol{k}_j, \qquad q_e = |\boldsymbol{Q}_e|. \tag{3.3}$$

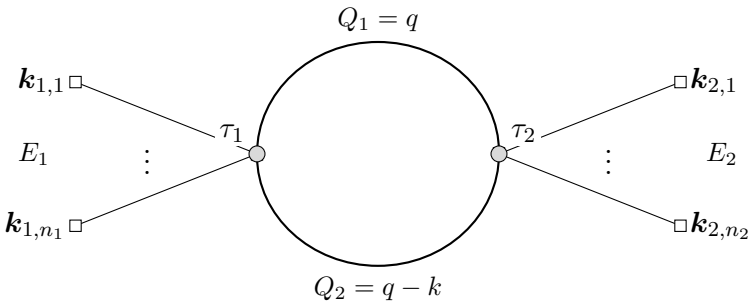
顶点的SK 符号决定传播子类型:

质量类型	顶点分支	指标包
massive	同分支$++/--$	{b[e],n[e,1],n[e,2]}
massive	异分支$+-/-+$	{b[e],n[e,1],n[e,2]}
massless	同分支$++/--$	{b[e],n[e,1],n[e,2]}
massless	异分支$+-/-+$	{b[e],0,0}

同分支传播子含theta ordering，称为full 传播子；异分支传播子没有theta 导数产生的shrink，称为cross 传播子。massive cross 仍有两个Hankel 顶点态；massless cross 是单个指数乘积，因此两个离散槽固定为零。表中给出cycle schema；fixed 传播子把第一槽b[e] 换成sentinel "F"。

3.3 Bubble 图作为输入例子

下图只是一个topology 输入例子，不代表生成器硬编码bubble。两条内部传播子可分别选为massive/massless full/cross；只需改变family metadata，统一积分Head 不变。



例如mixed bubble 可取$Q_1 = q$ 为massive，$Q_2 = q - k$ 为massless。pure massless bubble 则把两条传播子都设为massless。顶点符号从++ 改成+− 时，两条传播子自动从full 变为cross；不是换一套积分Head。

4 按mode 固定的三参数积分表示

所有top/sub-sector 都使用三个正式槽位

$$\boxed{J[\{a_v\}, \{\mathrm{pack}_e\}, \{n_{\mathrm{isp},r}\}]} \equiv \texttt{J[aList,linePacks,ispList]}. \tag{4.1}$$

三个槽位分别是

1. `aList`：当前sector 的独立时间变量幂次；
2. `linePacks`：按固定传播子槽位排列的传播子/缩并传播子指标包；
3. `ispList`：ISP 的正式指标槽，不是附注metadata。

式 (4.1) 是full 模式的schema。time-only 公开schema 为

$$\boxed{J[\texttt{sectorKey}, \texttt{timeShifts}, \texttt{stateBits}]}. \tag{4.2}$$

`sectorKey` 是root propagator 顺序的定长字符串；`timeShifts` 按当前sector 的compact vertex components 排列；`stateBits` 是n_i 类离散函数态，不是第二套sector key。初始化一旦选择mode，公开入口就只接受对应schema。

实际幂次包含初始化零点：

$$A_v = a_v + a0_v, \qquad B_e = b_e + b0_e, \qquad B_e^S = bS_e + bS0_e. \tag{4.3}$$

质量、ν_e、顶点、SK 符号、动量表达式、顶点能量、外不变量名称和symmetry rules 都不写入J。

theta 导数缩并传播子时，pack 变为

$$\mathrm{pack}_e = \{bS_e\}. \tag{4.4}$$

delta 函数合并两个时间变量，所以`aList` 只保留一个代表顶点槽；原顶点到compact slot 的映射保存在`sectorMetadataList`。传播子槽位始终保留，不能删除bS_e。

5 Massive 导数、EOM 与缩并

5.1 顶点导数与即时EOM

massive full/cross 的两个顶点态均取$n_{e,r}=0,1$。time 导数先产生

$$\partial_{\tau_r}: \qquad -J[\ldots,\{b_e-1,n_{e,r}+1,\ldots\},\ldots]. \tag{5.1}$$

若$n_{e,r}=1$，二阶态必须立刻EOM。对h：

$$J[n_{e,r}=2]=-J[n_{e,r}=0]-(2\nu_e+1)J[a_r-1,b_e+1,n_{e,r}=1]. \tag{5.2}$$

对裸H：

$$J_H[n_{e,r}=2]=-J_H[n_{e,r}=0]-J_H[a_r-1,b_e+1,n_{e,r}=1]+\nu_e^2 J_H[a_r-2,b_e+2,n_{e,r}=0]. \tag{5.3}$$

另一顶点指标不变。最后一项就是ν_e^2/x^2 二次pole；loop-momentum 导数若产生$n=2$，同样在seed 层立即使用对应EOM。当前实现由统一函数系统编译这些递推；旧版内置公式只作为历史基线。

5.2 通用2×2 函数系统输入

当前实现对每条massive 传播子输入

$$f''+P(x)f'+Q(x)f=0, \qquad \mathbf{F}=T(x)(f,f')^T, \qquad x=-\xi\tau, \tag{5.4}$$

其中P 是一阶导系数、Q 是零阶系数，T 缺省为单位矩阵；还必须给原始导数基底的完整Wronskian W。初始化层统一生成

$$A_0=\begin{pmatrix}0 & 1\\ -Q & -P\end{pmatrix}, \qquad A_T=T'T^{-1}+TA_0T^{-1}, \qquad W_T=\det(T)W. \tag{5.5}$$

IBP 的time/radial 导数只调用由A_T 编译的移位数据，theta shrink 只调用由W_T 编译的数据；不在IBP 层重算P,Q,T,W。缺省massive 模式是h，直接取$P_h=(2\nu+1)/x$、$Q_h=1$、$T=I_2$ 与完整W_h，所以$W_T=W_h$。裸H 是另一个$T=I_2$ preset。若从H 输入经一般基变换构造h，则

$$\mathbf{h}=x^{-\nu}\begin{pmatrix}1 & 0\\ -\nu/x & 1\end{pmatrix}\mathbf{H}, \qquad W_h=x^{-2\nu}W_H. \tag{5.6}$$

该编译接口已实现；显式W_T 只作$W_T=\det(T)W$ 的一致性校验。缺省h、裸H、一般T和非法输入由函数系统专项检查覆盖，h/H 另有独立物理benchmark。Wronskian 的标准推导见技术笔记专门subsection。

用户输入一般函数系统时，在对应massive 传播子中加入

```
"functionSystem" -> <|
  "variable" -> x,
  "P" -> Pexpr,
  "Q" -> Qexpr,
  "T" -> {{t11,t12},{t21,t22}},
```

```
  "W" -> Wexpr,
  "WT" -> Automatic,
  "shrinkBShift" -> 1,
  "shrinkZeroPointShift" -> zeroShift
|>
```

也可在放入输入记录前调用

```
compiled = compileFunctionSystem[functionSystem];
compiled["status"]
compiled["AT"]
compiled["WT"]
```

检查编译结果。`T` 省略时为单位矩阵；输入必须通过可逆性、Abel/Wronskian 一致性和有限Laurent 编译门禁。`shrinkBShift` 与`shrinkZeroPointShift` 是由函数系统和Wronskian 决定的物理编译数据，不是普通显示或basis 选项。使用内建h/H preset 时应保留缺省值；一般函数系统也应优先让编译结果决定这两个量。

5.3 Massive theta shrink

只有massiveFull 且$n_{e,1}+n_{e,2}=1$ 时有Wronskian shrink。顶点r 的系数为

$$-\chi_e(-1)^{n_{e,r}}\mathcal{C}_e, \qquad \chi_e=\begin{cases}+1, & ++,\\ -1, & --,\end{cases} \qquad \mathcal{C}_e=\frac{4i}{\pi}e^{\pi\,\mathrm{Im}\nu_e}. \tag{5.7}$$

其中χ_e 只由两个顶点的`vertexType` 派生，用户不能覆盖。child sector 的normalization 吸收$\chi_e\mathcal{C}_e$；因此normalized contact 中contour sign 与Wronskian 常数各消去一次，但有序Wronskian 的整体负号保留；不会在raw contact 之后重复乘branch sign。

对h 模式：

$$a_{\mathrm{merged}}=a_u+a_v-1, \qquad a0_{\mathrm{merged}}=a0_u+a0_v-2\nu_e, \tag{5.8}$$

$$bS_e=b_e+1, \qquad bS0_e=b0_e+2\nu_e. \tag{5.9}$$

对H 模式整数移位相同，但两个$2\nu_e$ zero-point shift 都为零。massiveCross 没有theta shrink。

因此现行缺省值及其实际child 映射为

模式	(s,z)	a_{merged}	$a0_{\mathrm{merged}}$	bS_e	$bS0_e$
h	$(1,2\nu_e)$	a_u+a_v-1	$a0_u+a0_v-2\nu_e$	b_e+1	$b0_e+2\nu_e$
H	$(1,0)$	a_u+a_v-1	$a0_u+a0_v$	b_e+1	$b0_e$

这里s 进入整数指标，z 进入zero point。对带圈动量的传播子，contact 支持$n_{e,1}+n_{e,2}=1$，故两种缺省模式都满足

$$b_e+n_{e,1}+n_{e,2}\equiv b_e+1=bS_e \pmod 2. \tag{5.10}$$

所以缺省h/H shrink 不改变subsector parity。h 的$2\nu_e$ 是zero point 而非整数seed 指标，H 的zero-point shift 为零；fixed/non-loop 传播子不进入parity 判定。对fixed/non-loop 传播

子不保留b/bS 指标槽，缺省z 进入target sector 的结构化prefactor，normalized contact coefficient 保留整数部分对应的模长幂r^{-s}。

一般不建议用户修改`shrinkBShift` 或`shrinkZeroPointShift`。若物理函数系统确实要求override，必须同时重新建立child-sector zero point、跨sector normalization coefficient 和parity metadata；只改一个shift 而沿用其它缺省数据会得到不一致的reduction/DE。若相对缺省的zero-point 重基不能精确判定为整数，相应sector 的parity 功能应关闭。需要额外吸收显式模长幂时，应通过后续master/basis 选择处理，而不是另改shrink shift。

6 Massless 双顶点quotient 与time-IBP

6.1 有序顶点定义

对有序顶点{`u`,`v`}，令$\Delta = \tau_u - \tau_v$。masslessFull 的同分支符号记为$\sigma = +1$ 对++、$\sigma = -1$ 对−−:

$$M_0 = \theta(\Delta)e^{-i\sigma q\Delta} + \theta(-\Delta)e^{+i\sigma q\Delta}, \tag{6.1}$$

$$M_1 = -\theta(\Delta)e^{-i\sigma q\Delta} + \theta(-\Delta)e^{+i\sigma q\Delta}. \tag{6.2}$$

第一顶点定义M_1 的方向。交换顶点时M_0 不变、M_1 变号。公开双顶点态记为$F_{n_1n_2}$，并用

$$F_{00} = M_0, \qquad F_{10} = M_1, \qquad F_{01} = -F_{10}, \qquad F_{11} = -F_{00} \tag{6.3}$$

定义quotient。正式J 始终保存三槽$\{b_e, n_{e,1}, n_{e,2}\}$，两个顶点指标均只取0, 1；relation 层以$n_{e,2} \to 0$ 为canonical 方向，不建立massless $n = 2$ 状态。

这里的$F_{11} = -F_{00}$ 已抽出每次顶点导数的$\pm i\sigma q$。原始时间导数关系仍为$\{11\} = +q^2\{00\}$：cycle 传播子由两次$b \to b-1$ 承载q^2，fixed/独立外动量传播子由导数算符显式乘两次模长参数。用户不需要为同一代数类重复生成source seeds；`DSSeeds` 只生成00/10，但导数产生的01/11 会连同这些系数立即约回。返回的`seedTemplateSummary["discreteStateAudits"]` 记录raw 四态数、代表态数、删除数和canonical direction。

每条massless 传播子仍保留自己的三槽pack；只有共享同一当前代表顶点对的time boundary 按共同theta 组合。fixed 传播子对应写成$\{\texttt{"F"}, n_{e,1}, n_{e,2}\}$。

6.2 两个顶点的regular 与delta 规则

由式 (6.2)，

$$\partial_{\tau_u} M_n = +i\sigma q\, M_{1-n} - 2n\,\delta(\tau_u - \tau_v), \tag{6.4}$$

$$\partial_{\tau_v} M_n = -i\sigma q\, M_{1-n} + 2n\,\delta(\tau_u - \tau_v). \tag{6.5}$$

对一般公开状态先应用式 (6.3)。在canonical representative $\{b, n, 0\}$ 上，对两个顶点分别求导的四个基本检查为

n	顶点	regular 指标变化与系数
0	第一顶点	$+i\sigma\,\{b-1,1,0\}$
0	第二顶点	$-i\sigma\,\{b-1,1,0\}$
1	第一顶点	$+i\sigma\,\{b-1,0,0\}$
1	第二顶点	$-i\sigma\,\{b-1,0,0\}$

同一顶点连续作用两次，regular 部分回到原canonical representative：

$$\partial^2_{\tau_u}: \quad -J[\{b-2,n,0\}], \qquad \partial^2_{\tau_v}: \quad -J[\{b-2,n,0\}]. \tag{6.6}$$

6.3 Massless shrink 与抵消

公开奇数顶点态$n_1+n_2=1$ 才产生theta-delta；在canonical representative 中即$\{n_1,n_2\}=\{1,0\}$：

$$\partial_{\tau_u}: \ -2\,\delta(\tau_u-\tau_v), \qquad \partial_{\tau_v}: \ +2\,\delta(\tau_u-\tau_v). \tag{6.7}$$

massless shrink 没有Hankel Wronskian prefactor，也没有2ν shift：

$$a_{\text{merged}}=a_u+a_v, \qquad a0_{\text{merged}}=a0_u+a0_v, \qquad bS_e=b_e, \qquad bS0_e=b0_e. \tag{6.8}$$

这里缺省compiled shift 是$(s,z)=(0,0)$。“没有shift”的准确含义是shrink 不新增任何整数幂或zero-point 幂吸收。对cycle 传播子，式 (6.8) 直接保留$bS_e=b_e$、$bS0_e=b0_e$。对fixed/non-loop 传播子，source sector 已有的模长幂必须保存在完整N_s；target 收缩后按其自身N_t 归一化。contact 一律先生成裸核系数c_{raw}，再写成

$$c_{\text{raw}}\,\frac{N_s}{N_t}\,J_t. \tag{6.9}$$

因此atomic fixed massless ++ 的source 与target 都含$sE1^{-\beta_1}$，比值为1，normalized contact 系数精确为-2。删掉target 中原有模长幂会错误得到$-2/sE1^{\beta_1}$。同一`sectorPrefactorData` 同时服务contact、`ds/DSDE` 与反向被积函数恢复，禁止在任一consumer 另猜一次模长幂。

massless 的subsector parity 不保持原offset。双顶点表示中contact 支持满足$n_1+n_2=1$，而$bS=b$，所以若parent 使用$b+n_1+n_2$ 分级，

$$b+n_1+n_2=b+1=bS+1 \pmod 2. \tag{6.10}$$

因此cycle massless contact 使child-sector parity offset 相对parent 翻转1；fixed/non-loop massless 传播子不参与parity，没有这次翻转。多传播子simultaneous contact 按被shrink 的massless cycle 传播子数模2 累加。package 不自动给pure massless family 启用parity；mixed h/H 系统一旦使用已认证root parity，massless cycle transition 必须携带上述affine offset，不能套用h/H 的“缺省不变”。

缩并后，剩余传播子的两个原顶点可能映到同一个active vertex。此时：

1. time 导数必须同时作用于两个顶点，不能只取第一个匹配；
2. 两个regular 顶点贡献按式 (6.5) 抵消；

3. 两个theta-delta 贡献的$-2/+2$ 系数相消，不再产生shrink 项；

4. 反对称态$n=1$ 在coincident 顶点恒为零。

这个零关系也必须作用于从top 方程产生的目标sub-sector 项。当前实现根据每个J 中已经变成单元素pack 的缩并传播子，重建目标sector 的顶点代表映射，再判定coincident $n=1$；不能只读取source top 的顶点映射。

对$+-$ 或$-+$ masslessCross，没有n、theta 或shrink。顶点regular 系数由该顶点分支决定：

$$+\text{顶点}:\ +i\,\{b-1\},\qquad -\text{顶点}:\ -i\,\{b-1\}. \tag{6.11}$$

6.4 同一顶点对多条full 传播子

同一当前代表顶点对的massive/massless full 传播子共享时间差Δ。写

$$G_e=\theta(\Delta)A_e+\theta(-\Delta)B_e, \tag{6.12}$$

则整个bundle 的boundary 只有

$$\delta(\Delta)\left(\prod_e A_e-\prod_e B_e\right). \tag{6.13}$$

定义$J_e=(A_e+B_e)/2$、$D_e=A_e-B_e$，可保持现有逐传播子pack：

$$\prod_e A_e-\prod_e B_e=\sum_{\substack{\varnothing\neq S\subseteq B\\ |S|\ \text{odd}}}2^{1-|S|}\prod_{e\in S}D_e\prod_{e\notin S}J_e. \tag{6.14}$$

一次事件可同时shrink 任意非空奇数条bundle 中的传播子，系数为$2^{1-|S|}$，但只合并一次顶点且只含一个delta。两条传播子只有single contacts；三条传播子另有系数$1/4$ 的triple contact。未选择的coincident full 传播子立即应用massless $n=1$ 为零和massive 顶点交换canonical，不会再次产生theta boundary。

sector 只允许连接两个不同当前代表类的contact 事件，按BFS 枚举；事件之间形成forest，但单个事件可选择多条平行传播子。单传播子的对称equal-time 值取$\theta(0)=1/2$，但不把它作为$\delta\theta^m$ 的点值乘法规则。保留逐传播子theta 时，统一Gaussian mollifier 给出的逐传播子积分在求和后严格等价于式 (6.14)。

例如三条同向masslessFull 传播子均取$n_1=n_2=n_3=1$，并从第一有序顶点求导。此时每条传播子的对称等时因子为零、contact difference 为-2，所以三个single-contact 项都因未选中传播子的等时因子为零而消失，只剩

$$\frac{1}{4}(-2)^3=-2. \tag{6.15}$$

指标上即

$$\begin{aligned}&J[\{a_u,a_v\},\{\{b_1,1,0\},\{b_2,1,0\},\{b_3,1,0\}\},\{\cdots\}]\\ &\quad\longrightarrow -2J[\{a_u+a_v\},\{\{b_1\},\{b_2\},\{b_3\}\},\{\cdots\}].\end{aligned} \tag{6.16}$$

三个pack 同时shrink，但只合并一次顶点且没有额外massless 幂次shift。共同-theta与Gaussian 逐传播子方案的完整推导、massive W_T 例子和等价条件见技术笔记附录。

7 Topology、标量积、ISP 与外腿能量

7.1 输入骨架

```
case = <|
  "name" -> "mixedBubble",
  "vertices" -> {
    <|"id"->v1,"vertexType"->"+",
      "externalLegEnergy"->Sqrt[sp[kE,kE]]|>,
    <|"id"->v2,"vertexType"->"+","externalLegEnergy"->E2|>
  },
  "lines" -> {
    <|"id"->1,"endpoints"->{v1,v2},"momentum"->q,
      "nu"->nuM,"massType"->"massive"|>,
    <|"id"->2,"endpoints"->{v1,v2},"momentum"->q-k,
      "massType"->"massless"|>
  },
  "loopMomenta" -> {q},
  "loopExternalMomenta" -> {k},
  "independentExternalMomenta" -> {kE},
  "ibpMode" -> "full",
  "ispData" -> {},
  "zeroPointRules" -> {
    a0[v1]->alpha1,a0[v2]->alpha2,
    b0[1]->beta1,b0[2]->beta2
  },
  "symmetryRules" -> {}
|>;
```

`endpoints` 对masslessFull 是有序输入；交换顺序会改变$n = 1$ 表示的符号。

输入后推荐立即通过公开初始化入口运行

```
context = DSInit[case, GenerateDerivativeMetadata -> True];
topo = context["topology"];
validation = context["validation"];
derivatives = context["derivatives"];
```

`validation["issues"]` 给出topology/ISP/函数系统问题；初始化metadata 同时保存sector、convention 和可选微分算符信息。

7.2 sp 与外不变量

用户端所有圈动量相关标量积统一写`sp[p,r]`。代码直接使用

```
SetAttributes[sp, Orderless];
```

实现$\mathrm{sp}(p,r) = \mathrm{sp}(r,p)$。这只是标量积交换性，不是图或积分族对称性。

用户可给圈动量和外动量任意符号名，例如`sah,bob,alice`。package 不从符号字符串猜测角色，也不再从一个统一列表自动分类；输入必须显式声明

```
loopMomenta
loopExternalMomenta
independentExternalMomenta
```

`loopExternalMomenta` 是圈积分外动量基，决定完整Gram 坐标、dqk 生成元和ISP 闭合维数；`independentExternalMomenta` 是topology 中实际无圈传播子/phase 的独立模长列表，只定义$|P_e|$，不定义彼此点积。package 不读取旧`externalMomenta/externalLegMomenta`；程序会从传播子routing 独立计算实际需要的空间并与用户声明比较，而不会替用户选择基。

本手册以后把`loopExternalMomenta` 的第i 项简称为`kLi`，把`independentExternalMomenta` 的第e 项简称为`kEe`。这是按两个输入列表的位置定义的角色名，不要求用户真的把动量符号写成`kL1/kE1`；`alice`、`bob` 等任意符号同样合法。两套编号各自从1 开始且互不共享：`kL` 第i,j 项在内部形成`qk[*,i]`、`kk[i,j]`，公开为`ssij`；`kE` 第e 项的模长平方在坐标/Jacobian 层使用`externalLegSquaredCoordinate[e]`，公开为`sEe`，不进入loop-IBP/ISP 的K。

结构圈数由内部无向多重图的第一Betti 数

$$L = E - V + C \tag{7.1}$$

给出；平行传播子逐条计数，自环各贡献一圈，`extLegs` 不计入E。逐条传播子删除后连通分量增加者为bridge，其余为cycle 传播子。`ibpMode->"full"` 还要求`loopMomenta` 数量等于L、routing 满秩并位于incidence cycle space；`ibpMode->"timeOnly"` 不生成momentum IBP，树图缺省即为该模式。

为消除任意圈动量平移，程序把传播子routing 写成$Q = Aq + r$，选满秩参考行A_R 后使用

$$q' = A_R q + r_R, \qquad r' = r - A A_R^{-1} r_R. \tag{7.2}$$

因此{`q+k1,q+k1+k2`} 实际只要求方向`k2`；加减号始终保留精确系数。这个计算只用于审计用户列表是否完备，不用于自动改写用户选定的动量基。

内部保留用户给定loop-external 基的平方Gram 原子，公开缺省为根号坐标：

```
kk[i,j] = sp[ki,kj]
ssij = Sqrt[sp[ki,kj]]
```

用户输出不保留内部`kk[i,j]`。

7.3 ISP 完备性

L 圈、K 个独立外动量共有

$$N_{\mathrm{sp}} = \frac{L(L+1)}{2} + LK \tag{7.3}$$

个圈相关独立标量积。用户定义的propagator 模长平方和ISP 必须构成可反解坐标。ISP 可以是一般标量积，例如

```
sp[l3,k321+l3]
```

而不只限于$q_i \cdot q_j$。package 验证闭合性，但不自动删传播子或替用户重选propagator basis。

每个ISP 使用`name/expr/range` Association，或等价三元组{`name,expr,range`}：

```
"ispData" -> {
  <|"name"->rho1,"expr"->sp[q1,k],"range"->{0,1}|>,
  <|"name"->rho2,"expr"->sp[q2,k],"range"->{0,1}|>
}
```

`name` 必须唯一，`expr` 使用已声明的动量基，`range` 给出整数指标seed 值域。ISP 的定义零点固定为0：正值是numerator 幂；用户显式给负值时表示该标量积坐标的额外denominator，package 不阻断。用户不需要也不能用输入改变这一零点。独立benchmark 与package 使用同一schema。

7.4 实际无圈模长与fixed 传播子

`independentExternalMomenta` 中每个表达式定义一个实际无圈模长。程序要求它们覆盖所有active 无圈传播子/被审计phase 中不能由完整loop Gram 表示的模长；不会从向量原子表自动拆分或补齐。

程序把实际无圈模长平方在完整loop Gram 与此前已选无圈平方坐标上按首次出现顺序做秩筛选，只为独立项建立模长变量：

```
sE1 = Sqrt[sp[PE1,PE1]], sE2 = Sqrt[sp[PE2,PE2]], ...
```

未入基的实际出现项保存为当前平方坐标的dependent binding；这些模长不进入loop IBP generator 或ISP closure。与两组动量都无关、只进入顶点相位的能量仍使用独立标量`ke[i]`。

例如

```
"loopExternalMomenta" -> {k1,k2}
"independentExternalMomenta" -> {kE1,kE2,kE1+kE2}
"lines" -> {...,"momentum"->kE1,...}
"vertices" -> {<|"id"->v1,"vertexType"->"+",
  "externalLegEnergy"->Sqrt[sp[kE1+kE2,kE1+kE2]]|>,...}
(* loop Gram: {ss11,ss12,ss22}; no-loop magnitudes: {sE1,sE2,sE3} *)
```

无圈模长只把整体反号$P \leftrightarrow -P$ 视为同一对象；|`alice+bob`| 与|`alice-bob`| 必须分别声明，不能合并。

坐标名只由两个用户列表各自的稳定顺序决定，与符号名无关。每套列表的总数为1–9 时使用`ss11/sE1`；10–99 时使用两位补零，例如`ss0101,ss0110,sE01`；100 起按该列表总数位数自然扩展。用户若需要其它坐标，统一通过`KinematicRules` 或`DSRedefineParameters` 给出：

```
DSInit[case,KinematicRules->{sp[k1,k1]->u^2,...}]
```

特别地，若topology 实际分别出现$|k_{E1}|$、$|k_{E1}+k_{E2}|$、$|k_{E2}|$，用户就应在`independentExternalMomenta` 中给出这三个表达式，缺省得到`sE1,sE2,sE3`；程序既不输出`sp[kE1,kE2]`，也不用余弦定理消去中间变量。若再声明$2k_{E1}$，它是过完备项并触发warning。

full 模式的`linePacks` 始终按root `lineOrder` 保留每条传播子的位置。cycle/fixed full pack 分别为`{b,n1,n2}` 与`{"F",n1,n2}`；收缩后分别为`{bS}` 与`{"F"}`。因此fixed 传播子没

有b/bS，但模长幂仍属于该sector 的结构化prefactor。time-only 的这些packs 只在Private producer 内存在，由中央adapter 映射到式 (4.2)。

每个sector 的完整normalization 写为结构化N_s。无圈模长部分按初始化时independentExternalMomenta 的稳定编号保存为kEpower[be1,be2,...]；当前参数表达式单独保存在kEParameterExpressions。例如用户重新定义参数后，一个编号模长可以映成若干新变量的组合，但幂向量和传播子/sector 身份不变；ds 根据表达式映射做链式法则，不能把kEpower 当成符号名猜测或只取一个单项式。已收缩massive 传播子的Wronskian normalization 按传播子编号保存于contractionParts，simultaneous contact 对每条传播子只累计一次。

初始化采用不阻塞脚本的两阶段接口：

```
proposal = DSKinematics[case];
(* 查看 proposal["defaultRules"] 与 proposal["selectionTemplate"] *)
custom = {sp[k1,k1]->u^2, sp[k1,k2]->u v,
          sp[k2,k2]->v^2+w^2, ...};
audit = DSKinematics[case, custom];
context = DSInit[case, KinematicRules->custom];
```

DSKinematics 的返回值分组列出：结构图/routing 审计、两个用户列表的exact/over/under 状态、基础坐标顺序、动力学规则矩阵与参数Jacobian、零空间和capability。主要Association key 包括

momentumDeclarationStatus, kinematicCoordinateStatus,
dependentMagnitudeBindings, missingDirections, missingMagnitudeSquares,
capabilities.

欠完备动量列表或动力学规则会以红色error 显示缺失方向/零空间并令DSInit 返回失败；所有后续模块读取同一capability，不能继续生成seed。过完备以红色warning 提醒，允许初始化和symbolic IBP seed，但关闭ds/DSDE 与唯一rep2innerform。对于过完备loop 声明，用户原列表保留作审计，实际n_K、Gram、dqk 与ISP 闭合只使用affine quotient 必要基effectiveLoopExternalMomenta；共同loop shift 不会虚增标量积空间。只有exact 状态同时开放初始化、time/momentum IBP、求导和反变换。

成功初始化后可直接查看或重定义notation：

```
notation = DSParameterNotation[context];
newContext = DSRedefineParameters[context, {
  sp[k,k] -> loopScale^2,
  sp[p1,p1] -> leg1^2,
  sp[p2,p2] -> leg2^2,
  sp[p1+p2,p1+p2] -> leg12^2
}];
```

新规则必须完备；返回的新context 有新的审计和inputHash，后续seed、ds 与DSDE 都读取它。规则右端可以是一般混合参数化，例如$(u+v)^2$；程序先展开每条右端并提取原子参数u, v，再建立完整Jacobian。参数数多于基础坐标数时属于过完备并关闭求导。正式函数名是DSRedefineParameters。

7.5 独立变量与微分方程求导

当前版本提供seed 层和表达式层的独立变量求导接口。变量分三类:

- **含圈传播子相关外动量模长**，缺省为`ssij`，通过外动量矢量导数组合实现。
- **实际无圈动量模长独立基**，缺省为`sE1,sE2,...`；从属模长先写成这些变量与`ssij`的binding。若它对应一条传播子，导数按binding Jacobian 同时作用于该传播子的分母和massive/massless building block。
- **独立顶点能量参数**，例如`ke[i]`；它不是动量向量，不定义点积。

对顶点能量y,

$$\frac{\partial}{\partial y}J=\sum_{v}\left[-\eta_v\,\frac{\partial E_v}{\partial y}\right]J[a_v\to a_v+1],\qquad \eta_v=\begin{cases}-i, & v=+,\\ +i, & v=-,\end{cases}\tag{7.4}$$

其中E_v = `vertexExternalEnergy[topo,v]`。额外负号来自本package 的时间幂convention $(-\tau_v)^{a_v+a0_v}$：相位对E_v 求导产生$\eta_v\tau_v=-\eta_v(-\tau_v)$。

对外不变量x_a，先引入外动量矢量导数

$$D_{ij}=k_i\cdot\frac{\partial}{\partial k_j},\qquad D_{ij}(k_m\cdot k_n)=\delta_{jm}(k_i\cdot k_n)+\delta_{jn}(k_i\cdot k_m).\tag{7.5}$$

然后在选定外不变量坐标$\{x_b\}$ 上解约束方程

$$\frac{\partial}{\partial x_a}=\sum_{ij}c_{ij}^{(a)}D_{ij},\qquad \sum_{ij}c_{ij}^{(a)}D_{ij}x_b=\delta_{ab}.\tag{7.6}$$

完整K^2 个D_{ij} 通常含零空间，解不唯一；当前实现默认使用上三角external-vector basis 取一组canonical 解，并在decomposition 数据中显式返回矩阵、系数、残差、`nullity`与`nonUniqueQ`。

相关接口为

```
makeExternalInvariantDerivativeDecomposition[topo, var]
applyExternalVectorDerivativeSeed[topo, int, gen]
applyExternalInvariantVariableDerivativeSeed[topo, int, var]
applyIndependentVariableDerivativeSeed[topo, int, var]
ds[expr, ssij, context]  (* context["topology"] 也可 *)
ds[expr, ssij]
```

`applyIndependentVariableDerivativeSeed` 会自动分派：loop Gram 变量先做上述decomposition，再把每个D_{ij} 作用到传播子、building block、ISP 和相位；实际无圈模长及其从属binding 使用该传播子的径向导数并微分相位；其它变量按独立顶点能量标量处理。一般用户坐标u 直接使用$\sum_a(\partial x_a/\partial u)\partial_{x_a}$，不会因$u$ 同时进入多个Gram 原子或binding 而提前返回。

`ds` 是表达式级总导数入口。变量必须使用函数族初始化后的外部名字；内部`kk[i,j]`不接受。若

$$\mathcal{I}(s)=\sum_r c_r(s)J_r(s)+c_0(s),\tag{7.7}$$

则

$$\partial_s \mathcal{I} = \sum_r c_r'(s) J_r(s) + \sum_r c_r(s) \partial_s J_r + c_0'(s). \tag{7.8}$$

程序先将J_r 替换为惰性token 求显式系数导数，再补上每个积分的指标导数，合并后统一执行EOM、symmetry 与canonical。它接受单个J 和任意线性组合，拒绝$J_i J_j$ 等非线性输入。

令$x_{ij} = \mathrm{sp}(k_i, k_j) =$ `ssij`^{2}。package 不重写平方Gram 导数，而是使用$\partial_{\texttt{ssij}} = 2\,\texttt{ssij}\,\partial_{x_{ij}}$ 调用原子层。显式系数与sector prefactor 仍执行完整乘积与链式法则；程序不使用`PowerExpand`。

对无圈massive 传播子e，记$B_e = b_e + b0_e$。径向导数包含$-B_e J[b_e \to b_e + 1]$；若A_T 的某个Laurent 项为$c\,x^p$ 并把顶点态α 送到β，则另有

$$c\, J[a_v \to a_v + p + 1,\ b_e \to b_e - p,\ n_{e,v} : \alpha \to \beta]. \tag{7.9}$$

这直接读取与IBP 相同的`derivativeTerms`，不读取只属于time-theta shrink的`WT/shrinkTerms`。

发布的examples 分两级：single-massive sunrise 只比较general IBP seeds 与general 参数微分算符；只有pure massive bubble 与mix bubble+tree 进入外部reduction 流程，其中massive bubble 继续承担reference/dlog/DE/scaling 对照。

对massive building block，external-vector/外不变量导数与qIBP、tIBP 自动使用同一条传播子上编译的同一份`compiledFunctionSystem`。普通导数只读取最终A_T 编译的`derivativeTerms`，因此用户选择h、裸H 或一般P, Q, T, W 后不需要为微分方程另设convention。$W_T = \det(T) W$ 及其`shrinkTerms` 只用于time-IBP 的theta coincidence/Wronskian shrink；普通动力学量导数不产生theta shrink，因而不调用W_T。

7.6 用户对称性规则

一般积分族对称性依赖质量和外参条件，当前实现不自动猜测一般图automorphism。程序会自动加入受门禁保护的原始self-loop tadpole 规则，并与用户`symmetryRules` 取去重并集：

```
repSymmetry0[topo_]
effectiveSymmetryRules0[topo_]
symmetry[expr_,topo_]
```

`repSymmetry0` 只返回用户原始规则；`symmetry` 对并集只做一次替换。自动规则只识别原始顶点相同的self-loop，不作用于cross propagator 或shrink 后才coincident 的普通传播子；odd ISP 归零还要求对应loop momentum 不被其它传播子共享。

8 IBP 生成元与公开接口状态

8.1 完整生成元

每个active vertex 有一个time generator。对L 个圈动量和K 个`loopExternalMomenta`，loop-momentum generators 为

$$\mathcal{O}_{\ell,v} = \frac{\partial}{\partial q_\ell^\mu} v^\mu, \qquad v \in \{q_1, \ldots, q_L, k_1, \ldots, k_K\}, \tag{8.1}$$

总数$L(L+K)$。不能漏掉$d/dq_\ell \cdot q_m$ 的交叉项或$d/dq_\ell \cdot k_j$。

当前核心使用`applyTimeGeneratorSeed` 与`applyMomentumGeneratorSeed`。微分方程方向的外部变量求导使用上一节列出的`applyIndependentVariableDerivativeSeed` 等接口。自动batch 必须在给定离散n 后调用，再立即EOM/canonical。

8.2 公开原子API

package 保留以下原子接口。推荐显式传入`DSInit` context 或parsed topology；若先调用`setIBPTopologyContext[topo]`，也可使用短签名：

```
dtau[i,expr,context]       or dtau[i,expr]
dqq[i,j,expr,context]      or dqq[i,j,expr]
dqk[i,j,expr,context]      or dqk[i,j,expr]
ds[expr,var,context]       or ds[expr,var]
rep2innerform[expr,context]
rep2outform[expr,context]
rep2Integrand[expr,context]
```

上式第三参数也可写成`context["topology"]`。resolver 会先识别完整context 并解包，不会把它误作原始topology case 再次解析。前三个复合算子分别表示$d/d\tau_i$、$d/dq_i \cdot q_j$、$d/dq_i \cdot k_j$，对J 的线性组合保持线性，并复用现有EOM、theta boundary与canonical。

例如先把离散态替换为字面0/1，再调用

```
int = makeBaseIntegral[topo] /. seedRules;
timeRelation = dtau[v1,int,topo];
qqRelation = dqq[1,1,int,topo];
qkRelation = dqk[1,1,int,topo];

setIBPTopologyContext[topo];
sameTimeRelation = dtau[v1,int];
clearIBPTopologyContext[];
```

短签名依赖唯一的已注册topology；同时处理多个topology 时应始终使用显式参数。

`rep2innerform`/`rep2outform` 转换用户动量与内部编号、`sp` 与$qq/qk/kk$、外不变量名和ISP 名。实现会先保护所有J，因此不改变三个指标槽、传播子pack 次序或任何$a/b/n$/ispN 值。

`rep2Integrand` 把J 展开为被积函数。普通时间幂、顶点相位、分母幂和ISP 直接相乘；每条未缩并传播子的非平凡Hankel/指数/theta block 用惰性`Hh[...]` 包装。它不积分、不生成IBP、不应用EOM。

9 Seed、linearData 与serializer

canonical seed 必须满足：

- 每个实际sector 都有全部active time 和$L(L+K)$ momentum generators；
- massive 无$n \geq 2$；

- masslessFull 只有$n = 0, 1$;
- 共同-theta odd-subset contact、contact 可达sector 与coincident canonical 已处理;
- 解析seed 保留非零$a0/b0/bS0$。

9.1 完整离散模板与连续指标撒点

`DSSeeds` 的`"allSeeds"` 是稳定的一维模板列表; 也可以用`DSAllSeeds[seedData]` 读取, 或用无参`DSAllSeeds[]` 读取最近一次成功结果。`"seedGroups"` 按缺省的`{sectorKey, ibpClass, generator}` 分组, 可由`DSSeedGroups` 读取; `DSSeedGroupMetadata` 返回同序来源说明。内部多重`Table` 在公开前用`Flatten[...,Infinity]` 磨平。每个模板已经对该sector 的massive 顶点遍历$0, 1$, masslessFull 则完整遍历两个quotient representatives $00/10$, 随后完成EOM/canonical; 连续a_i, b_i, bS_i 和ISP 指标仍保持general。该代表态缩减是已证明的代数canonical, 不是离散抽样, 也不是parity 筛选。连续指标交给`DSMetaSeedRange/DSGenerateIBP` 统一撒点。若EOM/canonical 把某个完整离散态化为精确零, 该密封模板仍作为合法恒等式保留在计数、哈希和离散态完整性审计中; 接口只拒绝缺失equation, 或非零且完全不含`J` 的伪模板。

先初始化逐组shift metadata:

```
groups = DSSeedGroups[seedData];
meta = DSMetaSeedRange[groups,{a[v1],a[v2],b[e1],b[e2]}];
```

若传入flat `allSeeds`, 整体统计为一组; 若传入nested seeds, 则每个顶层元素为一组, 组内任意深度先完全`Flatten`。因此用户可自行组织seeds, 不要求使用缺省生成元分组。声明指标相对seed 中的实际变量多报或少报都会双语warning, 但metadata 按实际完整集合继续初始化; 再次调用会覆盖此前状态。

统一最终关系包络写成

```
ibp = DSGenerateIBP[allSeeds,{-1,1}];
```

表示所有root 连续指标在生成后的IBP 关系中都必须落在整数闭区间$[-1, 1]$, 不是直接令seed 指标遍历该区间。对某组某指标的shift 集合Δ, 程序自动使用$[-1 - \min\Delta, 1 - \max\Delta]$ 反推较窄的seed 点域。逐指标精细包络写成

```
ibp = DSGenerateIBP[
  allSeeds,
  {a[v1],-1,1},{a[v2],0,1},
  {b[e1],0,2},{b[e2],-1,1}
];
```

精细形式必须exact cover 模板中出现的全部root 连续指标。未知、遗漏、重复、非整数边界、反向区间以及把`n[...]` 写进范围都会给红色双语Error。对ISP 指标, 自动反推的下界还会与用户target 下界取较大者; 因此package 不会为了覆盖升幂项自行向更负方向扩张。用户若在统一或精细形式中显式给出负ISP 下界, 该下界仍完整有效, 不会触发门禁。指数为0 的ISP 自身求导先精确化为零, 不产生由package 自己引入的-1 项。失败输入以相应结构化字段返回:

```
unknownIndices, missingIndices, duplicateIndices
invalidRanges, discreteIndicesInRangeSpec
```

`n_i` 从来不是连续撒点变量；若外部模板仍含符号`n[...]`，接口认为离散态或EOM阶段未完成并拒绝。成功结果的`status` 与`dSIBPStatus` 都是`"generated"`，并保存

```
templateSetHash, templateIntegrityAudit, continuousIndices
targetEnvelopeRules, derivedSeedRangeAudits
emptySeedRangeGroups, equationCount, equations
```

其中`equations` 是一维记录列表；每项同时带展开后的`equation`、原模板ordinal/hash、连续取值规则、sector/source/generator、离散态和`eomCanonicalQ/forbiddenNData` 证书，而不是失去来源信息的裸等式列表。`DSLinear` 直接消费整个成功Association。package 不设置方程数、连续规则数、离散态数或sector 数量上限；有限用户范围按所选截断策略完整生成。运行时间和内存由target 范围、截断选项及topology共同决定。

`SeedEnvelopeCut` 控制反推后的seed 点域。以下三种调用分别表示关闭截断、使用缺省策略和显式自定义参数:

```
ibp0 = DSGenerateIBP[allSeeds,{-1,1},SeedEnvelopeCut->None];
ibp1 = DSGenerateIBP[allSeeds,{-1,1},SeedEnvelopeCut->Automatic];
ibp2 = DSGenerateIBP[allSeeds,{-1,1},SeedEnvelopeCut->{10,4,5,4}];
```

设一个seed 点中全部正连续槽之和为C_+，全部负连续槽绝对值之和为C_-，$T = C_+ + C_-$。当$C_+ \geq C_-$ 时要求$T \leq Ex_1$ 且$C_- \leq Ex_2$；否则要求$T \leq Ex_3$ 且$C_+ \leq Ex_4$。缺省对含N 个连续槽的组使用$\{N+3, 4, 4, 4\}$。显式四元列表的每一项都可写整数或`Automatic`，后者只替换对应缺省项。筛点先于parity 和模板代入，返回的`seedEnvelopeCut` 字段保存解析参数、保留/跳点数和空组信息；`None` 恢复不截断的点域。

`TotalPowerLimit` 是独立的canonical 支撑截断，不改变seed 点域:

```
ibp3 = DSGenerateIBP[allSeeds,{-1,1},TotalPowerLimit->7];
ibp4 = DSGenerateIBP[allSeeds,{-1,1},
  SeedEnvelopeCut->{10,4,5,4},TotalPowerLimit->7];
```

给整数m 时，程序先合并同积分项并消去零系数，再只保留实际支撑全部位于逐槽target 盒、且每个支撑的连续指标绝对值总和不超过m 的方程；缺省`None` 关闭。它与`SeedEnvelopeCut` 可独立或联合使用，结果分别记录`totalPowerLimit` 和`seedEnvelopeCut`。full 模式的模板分解与canonical 缓存只处理Association 密封模板，裸表达式仍走原慢路径；这些优化不改变选项语义。

两种截断都不替代target/basis 闭合检查。`completeSystemQ` 描述所选包络与策略内的生成完整性；若更换target、master basis 或截断参数，仍须在外部reduction 后逐目标确认右端只含指定masters。需要复现未截断关系集合时，必须显式写`SeedEnvelopeCut->None` 和`TotalPowerLimit->None`。

枚举前逐输入组打印

```
编号 1（来源 ...）：{指标,min,max},...
Group 1 (source ...): {index,min,max},...
```

编号只表示用户输入分组，不假定它一定是一个IBP 算符。若反推区间出现min > max，warning 会说明目标包络窄于该组shift 跨度，因而不存在能使全部移位后积分留在包络内的seed 点；该组IBP 撒点为空，最终关系集合缺组，不能作为完整约化系统。该组保存在`emptySeedRangeGroups`，相应complete capability 为`False`。删除、改写或重排任何密封模板都会令`templateIntegrityAudit` 失败；仅改变外层`Table` 嵌套而保持flatten 后的同序模板不影响结果。`DSMetaSeedRange` 是唯一公开的逐组shift metadata 入口；后一次合法调用覆盖最近一次逐组范围状态。

所有运行时info、progress、Warning 与Error 按句先中文、后英文；`DSMessagesOff[]` 只关闭可选提醒，fatal Error 始终保留。

9.2 `timeOnly` 的唯一公开表示

`DSSeeds` 对所有`timeOnly` family 只返回

```
J[sectorKey_String,timeShifts_List,stateBits_List]
```

并记录`representation -> "J[sectorKey,timeShifts,stateBits]"`。`sectorKey` 的第e位按root 传播子顺序取full/shrunk 的1/0；top 是全1 字符串，前导零不可丢失。`stateBits` 的registry 同样按root 传播子顺序冻结：massive 传播子每顶点一位，massless quotient 整条传播子共享一位，cross/exponential 与缩并传播子不留占位。这条路线没有momentum generator、b/bS 或ISP，但fixed 传播子的模长幂仍在`sectorPrefactorData` 中参与`ds` 的乘积法则。用户不输入也不能调用Private 传播子pack 或vertex basis。

例如atomic massless 同分支family 的公开流程为

```
context = DSInit[case, RegisterAsCurrent->False];
seeds = DSSeeds[context];
linear = DSLinear[seeds, context];
```

其中top sector 的公开shared state 为$\{0,1\}$；raw $(n_1,n_2)\in\{0,1\}^2$ 只用于quotient 与导数审计。同分支单传播子可达sector keys 为`{"1","0"}`，异分支只有`"1"`。两个顶点各自的time generator 作用于top representatives；同分支shrink sector 另有合并顶点的time seed。成功时`DSSeeds` 与`DSLinear` 都返回`status -> "generated"`。含massless full 传播子的`DSTreeSeeds/repIterative/DSTreeNaiveIBP/DSTreeNaiveDE/DSTreeDLogDE` 当前返回`PendingRederivation` 与`masslessQuotientFormulaNotCertified`；表示adapter 不绕过这项数学门禁。

`linearData` 是后端中立的Mathematica Association，不是reduction 结果。核心字段为：

```
linearEquations, integralList, integralRules
sectorMetadataList, readiness reports
```

全sector 主积分候选必须统一排序，不能按sector 依次追加。

serializer 只转换`linearData` 的编号和语法。Kira serializer 默认只写基础输入、映射和metadata，包括`ibp.kira`、`list` 与`jobs.yaml`。它不保存本机Kira/Fermat 路径，也不运行Kira。

连续指标范围不属于topology。`DSSeeds` 只生成general templates；统一或逐指标范围都在`DSGenerateIBP` 调用中显式给出。新公开闭环只使用以下接口：

```
context = DSInit[case];
templateData = DSSeeds[context];
allSeeds = DSAllSeeds[templateData];
ibpData = DSGenerateIBP[allSeeds, {-1,1}];
linearData = DSLinear[ibpData, context,
  LinearSystemMode -> "symbolic"
];
```

普通batch 与`allSeeds` 都完整遍历独立离散基底；masslessFull的raw 四态审计与source representatives 计数分开保存。`DSSeeds` 的每条模板保存独立`templateHash`; `DSGenerateIBP` 的产物合同包含`sealedQ/sourceDigest/completeSystemQ/subsetQ/auditLevel/capabilities`。
所选包络与截断策略内的密封完整系统可进入formal 后端流程；密封子集和raw 输入可用于局部线性检查，但`completeSystemQ=False`，缺省`DSKiraExport` 与formal `DSKiraPlan` 必须拒绝。seed 或裸模板不能直接交给serializer；必须先得到成功的backend-neutral `linearData`。

`DSGenerateIBP`、`DSLinear` 与tree 侧的`DSTreeNaiveIBP`、`DSTreeNaiveDE`、`DSTreeDLogDE` 共享`AuditLevel`。缺省`AuditLevel->"standard"` 只消费同源sealed producer 的状态、计数、摘要和digest 字段，不重算模板/关系内容hash，也不重复canonical、parity、coverage 或tree residual 扫描。需要开发、独立检验或发布证书时显式使用`AuditLevel->"full"`。这不会放宽用户输入、未推导公式、递推终止、外部artifact 身份或reduction/DE closure 边界。

9.3 Kira 两阶段计划

`DSKiraPlan[linearData,spec]` 只在内存中构造计划并返回，成功状态为`"planned"`；它不运行Kira，也不写任何文件。计划里记录的`outputDirectory` 只是传给后续步骤的参数，实际写出Kira 输入目录的是`DSKiraExport`：它既接受该计划（`DSKiraExport[plan]`），也接受backend-neutral `linearData`。`spec` 必须显式给出`"stage"`，并可使用

```
preferredIntegrals, candidateIntegrals, activeBasis
numericStage, coefficientRules, outputDirectory, jobOptions
```

积分既可写成`J`，也可写成当前`linearData["integralList"]` 的一基编号；未知项会被丢弃，若候选因此为空则失败。

`linearData["integralList"]` 是唯一积分顺序。plan 和exporter 都不会再次排序；如需改变编号，必须在构造用户basis 前显式调用一次

```
linearData = DSReorderIntegrals[linearData, orderedJOrIDs];
```

该调用会同步积分表、映射和方程ID。`preferredIntegrals` 只记录候选偏好，不改变编号。

预约化阶段按既定顺序冻结用户指定的候选targets:

```
externalRoot = Environment["DSIBP_KIRA_WORKSPACE"];
preWorkspace = FileNameJoin[{externalRoot,"pre_reduction"}];
```

```
prePlan = DSKiraPlan[linearData, <|
  "stage" -> "preReduction",
  "preferredIntegrals" -> preferred,
  "candidateIntegrals" -> candidates,
  "coefficientRules" -> {},
  "outputDirectory" -> FileNameJoin[{preWorkspace,"kira"}],
  "jobOptions" -> Automatic
|>];
preExport = DSKiraExport[prePlan];
```

成功的`prePlan` 状态为`status->"planned"`，主要字段是

```
integralOrder, orderingConvention, preferredIntegrals
targetIntegrals, targetCount, numericStage, phaseIsolation
```

`preExport` 成功时状态为`status->"ready"`，并附带`dSIBPExportManifest` 与实际写出的`dSIBPExportManifestPath`。预约化结果用于外部backend 识别候选masters；它不是正式DE reduction。

用户取得并检查预约化master 后，用package 建立有序`userMI` basis:

```
linearWithUserMI = DSUserMI[linearData,userExpressions,<|
  "names" -> activeNames,
  "activeIndices" -> activeIndices,
  "derivativeVariables" -> {ss11,P0},
  "scalingDegrees" -> activeDegrees
|>];
userMIData = DSUserMI[linearWithUserMI];

formalWorkspace = FileNameJoin[{externalRoot,"formal_reduction"}];

formalPlan = DSKiraPlan[linearWithUserMI, <|
  "stage" -> "formal",
  "activeBasis" -> Automatic,
  "numericStage" -> "symbolic",
  "coefficientRules" -> {},
  "outputDirectory" -> FileNameJoin[{formalWorkspace,"kira"}],
  "jobOptions" -> Automatic
|>];
formalExport = DSKiraExport[formalPlan];
```

上述两个workspace 都必须位于当前Git 仓库之外；Kira 在WSL 中进入对应目录运行。仓库examples 不保存这两个工作树或它们的中间产物。`userExpressions` 的每一项可为单个`J` 或齐次`J` 线性组合。`DSUserMI` 检查全体/active 秩，在实际support 内保存`userMI[i]->J` 与pivot `J->userMI[i]+spectator J` 的可逆规则；它不把较小basis 宣称为全局积分空间的方阵变换。basis 附加后不能再重排。formal 阶段先对每个active expression 解析构造一阶导数和全部derivative target closure；只有闭合成功才返回

```
activeBasisPreview, analyticDerivativeCertificate
targetIntegralIDsPreview, minimalTargetsQ -> True
```

numericStage 缺省为"symbolic"。若显式选择"postDerivative"，数值规则也只能在解析导数及closure 已冻结后作用；formal plan 会冻结并实际复用同一份preparedLinearData，不得在plan 与export 之间重建另一份线性系统。export manifest 会保留解析证书、数值规则审计，以及equations/map/targets/rules/active payload和实际导出文件的SHA-256 artifactIdentity；DSKiraImport 按内容身份复核，不能用相同版本字符串替代artifact identity。两个阶段必须使用不同输出目录，不能让预约化manifest、targets 或reduction 覆盖正式结果。active-basis 导数内部始终把显式系数与裸J 分开；EOM、顶点/指标移位和canonical 只作用于裸积分，再乘回系数。若导出前仍残留内部执行helper，DSKiraExport 会拒绝写出，避免manifest 在保存和重新读取时发生二次求值。

稳定失败原因如下；任何失败计划都不能传给DSKiraExport：

```
notLinearData, invalidStage, invalidOutputDirectory
emptyCandidateIntegrals, missingActiveBasis
activeBasisClosureFailed, invalidNumericStage
```

DSKiraExport[plan] 仍会执行seed、linearData、active basis、targets 与微分变量数值化的全部serializer 门禁；成功只表示输入已写好或内存数据已就绪，不表示external reduction 已运行。

10 标准package 与初始化

一个example 的开头分为四个独立单元：package 加载、详细输入、缺省option、初始化。版本目录内Examples 通过公共loader 调用标准Needs 入口：

```
exampleDir = DirectoryName[$InputFileName];
Get[FileNameJoin[{exampleDir,"..","load_current_package.wl"}],
  CharacterEncoding->"UTF-8"];

(* 详细输入：vertices, lines, momenta, ISP, zero point,
   symmetry, parity 和 independent variables。 *)
case = <| ... |>;

(* 缺省选项：WriteInitializationFiles->False;
   GenerateDerivativeMetadata->False; OverwriteInitialization->False。 *)

init = DSInit[case,
  InitializationDirectory->FileNameJoin[{exampleDir,"init"}],
  WriteInitializationFiles->True,
  GenerateDerivativeMetadata->True
];
```

init/ 中的manifest.wl、topology.wl、sectors.wl 和conventions.wl 是可重新Get 的Association。derivatives.wl 缺省不生成；只有显式打开GenerateDerivativeMetadata 时才写出。已有目录若对应不同输入哈希，缺省拒绝覆盖。

初始化还会检查时间IBP 的小t 端点。程序在全部contact-reachable sector 中逐顶点使用已知h/H 领头幂次表，枚举每个入射massive 传播子的$n = 0, 1$ 与两支行为；任一Gamma 参数α 可被精确证明为整数时，未正规化family 会被拒绝，提示中列出原始顶点编号和α。正整数同样拒绝，因为整数指标移位最终会到达非正整数极点。此时应在相应`a0[vertex]` 中加入generic 符号正规化参数后重新初始化；程序不尝试用normalization 或其它发散项做形式抵消。含generic regulator 的表达式保持未定并允许继续。

`DSSeeds` 始终完整枚举物理独立的离散基底：massive 传播子为顶点$n_i = 0, 1$ 四态，masslessFull quotient 为shared bit 的二态；raw 四态relation 仍可由metadata 与导数检查审计。连续指标仍保持符号，只有后续`DSGenerateIBP` 的显式范围才生成整数点。timeOnly 与full 共用Head `J`，但使用式 (4.2) 与式 (4.1) 两个不兼容schema；不存在需要用户填写的`TreeState` 或其它私有分派参数。

初始化后可查看当前摘要、完整context 和公开接口清单：

```
DSInfo[]
DSInfo[context,"Full"]
DSPublicAPI[]
```

`DSPublicAPI[]` 返回当前版本、按工作流分组的全部公开函数和高层函数的实际`Options`；它是本附录汇总表与成品example coverage manifest 的共同核对入口。用户代码不应调用手册未列入该清单的私有helper。

11 运行消息与进度

package 使用一个集中消息层，所有耗时模块只上报结构化事件，不在内部循环散落`Print`：

```
DSMessagesOn[]
DSMessagesOff[]
DSMessagesQ[]
```

缺省开启info、progress 和非致命warning。关闭后隐藏这三类提醒，但fatal error 永远不能静默；函数返回值中的`status/issues` 仍是权威状态。warning 在notebook 中使用橙色，error 使用红色粗体，同时都发出标准Mathematica `Message`，使headless 日志也能捕获。

sector 初始化、偏导算符生成、seed、linearization、Kira 导出/导入验证、逐master/变量构造DE 和scaling check 都应给出阶段开始/结束信息。

Notebook 前端使用`Monitor` 配合`ProgressIndicator[n,{0,m}]`，或用单个`PrintTemporary` 配合`Dynamic`，在原位显示`n/m`。没有FrontEnd 时只打印开始、结束及大约每10% 的稀疏里程碑，禁止每处理一项增加一行。总数不能预先可靠获得的阶段只报告阶段状态，不伪造`n/m`。

12 Kira 结果取回到微分方程

package 只生成Kira 输入，不运行reduction。完整闭环为：

```
externalRoot = Environment["DSIBP_KIRA_WORKSPACE"];
workspace = FileNameJoin[{externalRoot,"pure_massive_bubble"}];

templateData = DSSeeds[init];
allSeeds = DSAllSeeds[templateData];
ibpData = DSGenerateIBP[allSeeds,{-1,1}];
linear = DSLinear[ibpData,init];
linear = DSUserMI[linear,userExpressions,<|
  "names"->activeNames,
  "activeIndices"->activeIndices,
  "derivativeVariables"->{ss11,P0},
  "scalingDegrees"->activeDegrees
|>];

formalPlan = DSKiraPlan[linear,<|
  "stage"->"formal",
  "activeBasis"->Automatic,
  "numericStage"->"symbolic",
  "coefficientRules"->{},
  "outputDirectory"->FileNameJoin[{workspace,"kira"}]
|>];
kiraExport = DSKiraExport[formalPlan];

(* 用户从 WSL 进入同一外置目录运行 Kira。 *)
kiraResult = DSKiraImport[
  FileNameJoin[{workspace,"kira"}],init
];
de = DSDE[kiraResult,{ss11,P0},
  OutputDirectory->FileNameJoin[{workspace,"results","dlogDE"}]
];
scale = DSScaleCheck[de,<|
  "relation"->"PureMassiveBubble",
  "variables"->{ss11,P0},"weights"->{1,1}
|>];
```

`DSIBP_KIRA_WORKSPACE` 必须指向本发布仓库之外。04 的脚本会拒绝把Kira 工作树写回仓库。WSL 只运行该外置目录中的`jobs.yaml`；完成后仍由Wolfram 从同一目录导入结果。04/06 目录中的`kira_input_summary.wl` 与`kira_result_summary.wl` 只保存正式检验的轻量记录，不是reduction fixture。

`DSKiraImport` 缺省只从`results/Tuserweight/kira_list.m` 与`results/Tuserweight/masters` 读取Kira family 输出；其它位置必须显式给出`KiraReductionFile/KiraMasterFile`。它检查export manifest 的内容身份、双向积分映射、master list、targets、系数域和完整reduction rules。任何结构化必需文件缺失、内容篡改或convention/parity 不一致都会阻止DE；`kira.log` 的完成标记只作warning 诊断，不单独否定已经闭合的结构化结果。根号代数系数可在Kira 侧映射为小写原子，manifest 保存可逆映射；导入后恢复原表达式。`DSDE` 对同序master list 调用`ds`，因

此动力学量系数和完整sector normalization N_s 的导数都自动包含在内：

$$\partial_x J_s = (\partial_x I_s)\, N_s + \partial_x \log N_s\, J_s. \tag{12.1}$$

实现中后一项按同一metadata 计算为`D[Log[N_s],x] J_s`，不能只微分裸积分核。约化后的内部`kk/ISP` 系数还会统一转回初始化声明的根号坐标。`rep2Integrand` 从同一metadata 把N_s 乘回具体被积函数；未约回master 的对象必须在结果中显式列出。

`numericStage->"symbolic"` 时，参与微分的变量直到`DSDE` 完成前都保持符号。`numericStage->"postDerivative"` 则先冻结解析导数和完整target closure，再把同一固定有理点代入待约化系数；它只证明该点的reduction 与DE，不得写成全变量矩阵证明。最终有理probe 必须先检查全部分母非零。交付的pure-massive-bubble closed-loop example 使用缺省root coordinate `ss11=Sqrt[sp[k,k]]` 和`derivativeVariables->{ss11,P0}`，不能把旧`s11` reduction 作字符串改名后充当新坐标的fresh 结果。

formal DE 若需要下述Kira 虚轴映射，必须先用符号`DSLinear` 构造`linearData`；待旋转的物理坐标及其从属Gram 原子只能在formal `DSKiraPlan` 的`postDerivative` 规则中给值。若这些对象已在`DSLinear` 的`CoefficientRules` 中被消去，导出会明确说明数值时序冲突并停止，因为此时无法保证方程与manifest 使用同一物理截面。只做普通numeric linear reduction、且不构造formal DE 时，仍可在`DSLinear` 代入精确规则。

Kira-only 虚轴运动学convention 只作用于serializer/importer。初始化从vertex-phase 与传播子metadata 识别顶点相位独立参数和fixed massless propagator 模长，并在进入Kira 前做`k->-I ik`；cycle 传播子的`xi` 是积分变量，不列为外部backend 坐标，复合对象按结构聚合，不按符号名字猜角色。fixed 模长不是独立符号时，程序会提示用`KinematicRules` 绑定独立坐标并停止导出。数值规则只给`ik` 精确实有理值。manifest 同时保存物理截面、来源顶点/传播子、双向映射、`D_k=I D_ik` 和Euler 算符不变性；公开物理API、IBP 与scaling convention 不改变。若该物理原子还定义了内部Gram 坐标，程序会先形成物理截面再按当前`KinematicRules` 重生内部规则。例如`kk[1,1]=loopScale^2` 且backend `loopScale->43/17` 时，物理截面为`loopScale->-43 I/17`、`kk[1,1]->-1849/289`；两者符号不一致时export 不得继续。所有剩余整体虚相位由可逆的逐积分phase gauge 消除；`ibp.kira` 只允许实有理函数，出现`I`、`Complex` 或`dsii` 时export 直接失败。初次reduction 探测应按预估master 规模设置有界targets，而不是选择完整积分表；formal workflow 只选择active basis 与其导数闭包。

`DSScaleCheck` 验证与reduction 数据范围相符的Euler 恒等式。写

$$\partial_{x_a} \boldsymbol{I} = A_a \boldsymbol{I} + \boldsymbol{s}_a, \qquad \mathcal{E} = \sum_a w_a x_a \partial_{x_a}, \qquad \mathcal{E} I_i = \delta_i I_i, \tag{12.2}$$

`DSScaleCheck` 根据manifest 物理规则实际固定的scaling variables 数量报告证书范围：固定零个时为`symbolic`，固定全部时为`exactPoint`，只固定一部分时为`fixedSection`；只有第一种返回`symbolicQ=True`。因此`postDerivative` 若只固定d, ν 等非scaling 参数，仍可返回全符号`matrixResidual/sourceResidual`；若scaling coordinates 已被固定，则定点或固定截面矩阵不能外推成全局符号DE。此时全局符号齐次性由active master 的外部Euler 导数与time/q-dilation IBP 恒等式在source level 单独认证，并与相应residual 分别通过。则同序、逐项齐次的master basis 应满足

$$\sum_a w_a x_a A_a = \mathrm{diag}(\delta_i), \qquad \sum_a w_a x_a \boldsymbol{s}_a = 0. \tag{12.3}$$

若换基为$\boldsymbol{I}' = T(x)\boldsymbol{I}$，则必须使用

$$A'_a = (\partial_{x_a} T)T^{-1} + TA_aT^{-1}, \qquad B' = \mathcal{E}[T]T^{-1} + TBT^{-1}, \quad B = \sum_a w_a x_a A_a. \tag{12.4}$$

因此依赖动力学量的normalization 不能只作矩阵共轭；$\mathcal{E}[T]T^{-1}$ 是scaling-degree 修正。对normalized master $M_i = N_i J_i$，其δ_i 还必须包含$\mathcal{E}[N_i]/N_i$。reference bubble中`ks*DEks0+P0*DEP00` 正是权重$\{1,1\}$ 的Euler matrix；应检查非对角元为零、对角元等于同序master degrees，且结果不依赖整体尺度。

首个闭环example 固定使用pure massive bubble reference 的$--$ branch 和even-parity closed subsystem，不生成全parity 大系统。reference 的顶点交换symmetry 只在$P_1 = P_2$ 成立。reference 的$V_\pm = 0$ 与package 的$--$ 分支使用相反的能量参数号，因此$P_{\mathrm{pkg}} = -P_{\mathrm{ref}}$；Kira-only 截面再取$P_{0,\mathrm{pkg}} = -i\,ip0$。独立$P_1/P_2$ family 不得复用该symmetry。

同目录`reference_user_mi_basis.wl` 只保存19 个有序候选（全部active，没有辅助关系）、physical `ks=ss11`、导数变量和scaling degrees；`main.wl` 必须把这些数据交给package `DSUserMI`，由package 构造秩、双向映射、backend ID 与derivative closure，不在example 中自写basis adapter。`dlog_basis.wl` 只保留reference-readable 记号。当前package 在`SeedEnvelopeCut->Automatic` 与`TotalPowerLimit->None` 下fresh 生成2442条canonical relations、2460 条formal equations 和1317 个源积分；300 个targets 全部约化，0 unreduced。target 包络为$a_v \in [0,3]$、$b_e \in [-3,5]$，覆盖19 项basis 及导数支撑。Kira 保留19 个active masters 和5 个额外全局项，目标RHS 与19 维DE 均闭合；`w10*1` 的外部驱动墙钟时间约为5 s。当前十九项basis 统一权重、直接在符号k_s 下建立physical dlog 矩阵，physical P_0、backend `ip0` 与physical k_s 三套19×19 矩阵均为361/361 精确相等、非零差值0（旧版第15–18 项的显式k_s 恢复不再存在）。reference 侧只复制并校验既有`full19_connection/DEP0.m`、`DEks.m`、`DEscaleCheck.m` 与`basis/MIdlogNote.m`，禁止重新生成reference IBP 或运行reference Kira。

13 Tree vertex family

Tree 使用原生time-only 公开对象：

```
J[sectorKey,timeShifts,stateBits]
```

`timeShifts` 按当前sector 的active vertex-component 顺序保存时间幂次；`stateBits` 按该sector 的registry 保存存活building blocks 的离散态。massive-only 论文公式需要的vertex basis 由package 在Private context 内按`sectorKey` 构造，完成迭代或dlog 计算后立即映回上述公开表示；用户结果、诊断、master list 和`linearData` 均不得出现一参数形式，也不得出现用内部传播子pack 表示的旧time-only `J`。

Loop 到tree 的投影使用完整物理传播子幂。目标项相对参考seed 的每条传播子显式能量系数为$k_e^{-\Delta(b+b0)}$；缩并传播子改用$bS + bS0$。source 与target 都按$(-\tau)^A$ 定义，contact 合并不另乘$(-1)^{\Delta\sum(a+a0)}$。当前sector 的$a0$ 进入tree 的ν_0。所以h-mode contact 的$bS = b + 1$、$bS0 = b0 + 2\nu$ 必须共同给出$k^{-2\nu-1}$，不能只保留整数k^{-1}。

单顶点p-fold family 的2^p 个masters 按二进制顺序`00...0,00...1,...,11...1` 保存。`repIterative[expr,end]` 使用2401.00129 Eq. (3.37)、(3.47)、(3.50) 直接生成的general-index 单次规则，把每个a_e 约化到指定终点；`Automatic` 表示全零终点，

长度不符或指标/终点不是可判定整数时拒绝。它不设置最大步数，而逐步验证字典序{剩余可contact 的theta 传播子数, $\sum_e |a_e - a_{e,\mathrm{end}}|$} 严格下降，并检测重复canonical 状态；不下降、递推奇异或循环才是数学失败。含contact source 时，raw 与sector-tagged 迭代都从`DSTreeSeeds` 的direct pure-time relation 生成同一首步，因而保留由用户传播子`momentum` 字段决定的模长；旧三槽loop 投影只作交叉验证。

`DSTreeDLogDE` 的sector 对角块使用该文Eq. (3.54)–(3.55)，非对角块使用Eq. (3.66)–(3.68)。letters 按顶点顺序逐块排列：每个顶点先列massive-leg energies，再按binary master order 列cut letters，最后稳定去重；`letterMatrices` 使用同一键顺序。G_{+-}/G_{-+} 没有theta，不使用Wronskian/contact；G_{++}/G_{--} 的contact source 先由已验证的loop `dtau`、compiled W_T、共同-theta和coincident canonical 处理，再映射到tree lower sector。

程序对每个binary master 保持n 不变，只把当前求导顶点设为$a = 1$，用所得$R^{(1)}$ 构造非对角contact selector；不会误用$a = 0$ seed 的$R^{(0)}$。每个lower sector 分别保存有理contact selector 与物理sector prefactor，后者吸收Wronskian 和zero-point 产生的连续能量幂；complete normalization 的对数导数由naive/direct 两条路线共同消费。以下返回字段用于审计：

```
masterNormalizationRecords, dlogSelectorCoefficients, selectorAudits, contactMaps
omegaBlocks, dlogQ
```

`masters` 中每项的`coefficient` 是对应sector 的有理selector S_s，不是完整物理normalization；N_s 与乘积S_sN_s 只在同键的`masterNormalizationRecords` 中给出。矩阵、letters 与该master 顺序完全一致。

还提供不使用上述直接dlog/递推矩阵的交叉路线：

```
ibp = DSTreeNaiveIBP[context, treeDLog["masters"]];
de  = DSTreeNaiveDE[ibp, {K1,K2,k12}];
```

`DSTreeNaiveIBP` 对每个sector 的binary master 和活动顶点生成$a_v = 1$ 的loop 代表元，只调用`dtau` 与既有contact canonical，再把投影方程作为有限线性系统直接求解；它不调用`repIterative` 或$A_\pm$。`DSTreeNaiveDE` 对外腿相位项复用loop 投影，对massive 传播子动量模长则直接使用h 的动量导数关系，因为loop 适配器中的内部动量是积分变量，不能冒充tree 的外部参数。对于bare sector integral I_s 的顶点态n，raw tree 导数为

$$\partial_k I_s[a, n = 0] = -I_s[a + 1, n = 1], \tag{13.1}$$

$$\partial_k I_s[a, n = 1] = I_s[a + 1, n = 0] - \frac{2\nu + 1}{k} I_s[a, n = 1]. \tag{13.2}$$

同一传播子的两个顶点分别贡献，随后统一用naive time-IBP 约到指定masters。程序依次使用$J_s = N_sI_s$ 与$F_s = S_sJ_s$，所以乘积法则完整加入$(\partial S_s)J_s$、$S_s(\partial \log N_s)J_s$ 和$S_sN_s\partial I_s$；只允许前两项产生参数与动力学量的有理对数导数，不把$k^{c\nu}$ 留在系数中。输出的`matrices/sources/residualObjects` 与输入master 顺序一致。缺省masters 来自`DSTreeDLogDE`；做独立检验时应显式传入列表，确保两路线只共享basis 和normalization，而不共享reduction rules。

若不同lower sectors 具有相同长度的`timeShifts`，公开`sectorKey` 仍唯一编码shrink set；内部按root 传播子顺序排列的pack pattern 只用于验证该key，不是第二套公开身份。公式型tree 路线当前只认证massive-only family；含massless full 传播子时五个tree 公式接口均fail closed 为`PendingRederivation`，直到quotient basis 的递推和dlog 公式重新推导完成。

14 常见错误与当前限制

- 把`sp` 的Orderless 当作图对称性：错误。
- massless 传播子交换顶点却不翻转$n = 1$ 表示：错误。
- massless $n = 1$ theta 导数漏掉$-2/+2$ shrink：错误。
- 对$\delta\theta^m$ 直接代入$\theta(0)^m = 2^{-m}$：错误；必须先合并共同theta 或统一正则化整个乘积。
- massive/massless shrink 都令merged a 减1：错误；只有massive Wronskian 有该整数移位。
- 把所有full 传播子的幂集当作sector：错误；只枚举contact 可达状态。
- top 方程的sub-sector 项仍用source 顶点映射canonical：错误，当前实现已按目标sector 修复。
- massive $++$ 与$--$ 使用同一contour sign，或让raw contact 与child normalization 使用不同sign：错误；两处必须使用同一个由顶点派生的χ_e。
- 在EOM 前保留符号n 或把$n = 2$ 留给后端：错误。
- shrink 后仍在J 保留两个独立a：错误。
- 把`ke[i]` 放入momentum generators：错误。
- seed 直接导出Kira：错误；先构造`linearData`。

当前未实现：一般图automorphism/参数对称性的自动发现、scaleless 全自动前端、自动选择propagator basis、streaming/chunked 高圈seed、massless 三槽quotient basis 上的公式型tree 递推/dlog，以及实际运行reduction。当前版本已实现标准context、动力学变量审计、h/H parity、交互消息、Kira 结果取回和loop/tree DE/scaling；package 仍只验证并导入用户在外部生成的完整reduction。

A 公开函数汇总

两个选项由多个命令共享，表中不再逐行重复：`ProgressReporting`（缺省`Automatic`）控制控制台进度输出，由`DSInit`、`DSSeeds`、`DSGenerateIBP`、`DSLinear`、`DSKiraPlan`、`DSKiraExport`、`DSKiraImport`、`DSDE`、`DSScaleCheck`、`DSRedefineParameters`、`DSTreeNaiveIBP` 与`DSTreeNaiveDE` 接受；`AuditLevel`（缺省`"standard"`）设置自审计深度，由`DSGenerateIBP`、`DSLinear`、`DSTreeNaiveIBP`、`DSTreeNaiveDE` 与`DSTreeDLogDE` 接受。第三列只列各命令专用的option 与spec 键及缺省；它们的语义和完整用法随对应命令在正文各节介绍。

函数	用途	专用option/spec 键与缺省
DSKinematics	提议或审计结构/routing、双动量列表、动力学变量规则、秩、零空间与可逆性	第二参数rules 缺省Automatic（缺省出提案）
DSInit	解析family、注册context、生成topology/sector/convention metadata	KinematicRules = Automatic; GenerateDerivativeMetadata = False; WriteInitializationFiles = False; InitializationDirectory = Automatic; OverwriteInitialization = False; RegisterAsCurrent = True
DSInfo	返回当前初始化摘要与readiness	无
DSParameterNotation	返回缺省/当前根号notation 与用户变量	context=当前已注册context
DSRedefineParameters	用完整新规则重新初始化参数坐标	无专用option
DSMessagesOn, DSMessagesOff, DSMessagesQ	分别开启、关闭或查询非致命提醒；fatal error 始终保留	无
DSPublicAPI	返回公开函数分组、清单与高层缺省options	无
DSSeeds	massive 完整枚举四态、masslessFull 枚举00/10 quotient representatives，并生成连续指标保持general 的一维模板allSeeds 与推荐分组；所有模式返回统一三参数J	无专用option
DSAllSeeds	从seedData 或最近一次成功调用读取扁平模板列表	无
DSSeedGroups, DSSeedGroupMetadata	读取缺省嵌套seed 分组及其同序来源metadata；也可读取最近一次成功结果	无
DSMetaSeedRange	按用户seeds 外层结构初始化逐组shift metadata；flat 为一组，nested 按顶层分组；再次调用覆盖旧状态	声明指标多报或少报只warning，按实际变量继续
DSGenerateIBP	从统一或逐指标最终关系包络反推逐组seed 点域，按选项筛选seed 与canonical 支撑；返回范围、截断metadata、方程记录和证书	SeedEnvelopeCut = Automatic（亦可None 或四元参数）；TotalPowerLimit = None（或整数）
DSLinear	seed 转后端中立linearData，其integralList 是唯一后端顺序	LinearSystemMode = "symbolic"; CoefficientRules = Automatic; KiraOrdering = Automatic
DSReorderIntegrals	在basis/backend 前显式同步重排积分表、映射与方程ID	order 为现有J 或ID 列表
DSUserMI	从有序J/齐次线性组合构造可逆support 坐标、公开userMI token 与导数闭包	spec 键：names （缺省active1、active2...）；activeIndices （缺省全部位置）；derivativeVariables = Automatic; scalingDegrees = Automatic
DSKiraPlan	按既定顺序生成预约化candidates，或formal userMI derivative closure/minimal targets	spec 键：stage 必填; preferredIntegrals = {}; candidateIntegrals （preReduction 阶段缺省全部积分）；activeBasis （formal 阶段缺省Automatic）；numericStage = "symbolic"; coefficientRules = {}; outputDirectory = None; jobOptions = Automatic

续下页

函数	用途	专用option/spec 键与缺省
`DSKiraExport`	从`linearData` 写Kira 基础输入与映射	`OutputDirectory` = None; `KiraTargetIntegrals` = Automatic; `KiraCoefficientRules` = {}; `KiraJobOptions` = Automatic; `KiraActiveBasis` = None; `KiraRequireCompleteSystem` = True; `KiraNumericStage` = `"symbolic"`
`DSKiraImport`	读回完整Kira reduction/master 输出	`KiraReductionFile`、`KiraMasterFile` = Automatic（缺省`results/Tuserweight/` 的family 输出，其它路径显式给出）；`KiraCompletionFile` = Automatic; `KiraCompletionPatterns` = Automatic
`DSDE`	用`ds` 与reduction 构造同序DE，并检查所有导数积分已约回master	变量参数缺省为全部初始化变量; `OutputDirectory` = None
`DSScaleCheck`	检查top/residual Euler scaling	`ScalingRelation` = `"Custom"`; `ScalingVariables`、`ScalingWeights`、`ScalingDegrees` = Automatic; allowbreak spec 键`relation`、`variables`、`weights`、`degrees` 与之等价
`DSTreeSeeds`	生成direct pure-time/tree sector rules；loop 投影只作交叉验证	无专用option （整体调用遍历全部time 生成元顶点，亦可单顶点调用）
`repIterative`	tree 积分迭代到指定a_e 终点；严格下降/循环/奇异性门禁保证数学终止	end 参数缺省`Automatic` （各顶点零时间幂）；无步数上限option
`DSTreeNaiveIBP`	联立投影后的tree time-IBP 并解一步升幂对象	masters 参数缺省为`DSTreeDLogDE` 同序master 列表
`DSTreeNaiveDE`	用naive tree IBP 构造同序normalized-master DE	variables 参数缺省为顶点能量与massive-leg 模长; allowbreak masters 参数缺省同`DSTreeNaiveIBP`；亦可直接接收其返回值
`DSTreeDLogDE`	生成tree dlog connection、letters 和同序masters	无专用option
`dtau`	指定顶点的time-IBP	显式topology 或当前context
`dqq/dqk`	指定loop generator 的momentum-IBP	显式topology 或当前context
`ds`	对带动力学系数的J 线性组合求总导数	变量必须是初始化外部名字
`rep2innerform`	用户/内部标量积表示转换，不改J 指标；无唯一逆映射时inner 明确失败	显式topology 或当前context
`rep2outform`		
`rep2Integrand`	把loop 指标积分展开成惰性被积函数	不积分、不生成IBP
`symmetry`	应用自动tadpole 与用户symmetry rules 的并集	单次应用，不自动重复
`repSymmetry0`	返回输入中的原始用户symmetry rules	无

表中的原子、tree 与`DS*` 高层接口均由当前主线提供。各函数的精确可用options以`Options[函数]` 为准；massless 公式型tree 接口以结构化`PendingRederivation` 明确失败，不得静默回退到历史表示或不完整结果。

dSIBP Handbook

1 Scope and quick start

dSIBP is a topology-driven Wolfram Language package for de Sitter Feynman integrals. It constructs time and loop-momentum IBP seeds, parameter-derivative operators, sector metadata, backend-neutral `linearData`, and Kira input files. It does not perform reduction internally. External reducers such as Kira or Rational Tracer remain separate steps controlled by the user.

The package supports arbitrary loop numbers and topologies, mixed massive and massless propagators, exact contact-sector construction, EOM and symmetry canonicalization, user-defined master bases, and differential-equation and scaling checks. The public topology schema is deliberately strict: package metadata is derived from the input graph rather than inferred from symbol names.

A standard session loads the package, initializes a family, generates symbolic seeds, expands continuous indices, and constructs linear data:

```
packageDir = FileNameJoin[{Directory[],"dSIBP"}];
If[!MemberQ[$Path,packageDir],AppendTo[$Path,packageDir]];
Needs["dSIBP`"];

context = DSInit[case];
seedData = DSSeeds[context];
allSeeds = DSAllSeeds[seedData];
ibpData = DSGenerateIBP[allSeeds,{-1,1}];
linearData = DSLinear[ibpData,context];
```

Use `DSKiraPlan` and `DSKiraExport` when Kira input is required, run Kira outside the package, and use `DSKiraImport`, `DSDE`, and `DSScaleCheck` only after complete external results are available.

The bundled examples cover six complementary workflows:

- `01_mixed_bubble_workflow.wl`: basic initialization, generation, linearization, integral ordering, and planning;
- `02_function_system_hankel.wl`: Hankel and related function-system input;
- `03_single_massive_sunrise`: general two-loop seeds and parameter operators, without sampled relations or reduction;
- `04_pure_massive_bubble_closed_loop`: a 19-master Kira/DE/reference/scaling closed loop;
- `05_tree_two_vertex_time_ibp`: direct pure-time/tree relations and naive versus dlog differential equations;
- `06_mix_bubble_tree`: mixed cycle/bridge propagators, custom coordinates, contact sectors, and an 81-master reference summary.

2 A complete mixed bubble-tree example

A useful nontrivial input contains a massive cycle propagator and a massless cycle propagator between v_1 and v_2, followed by a massless fixed bridge from v_2 to v_3. Only the two cycle propagators carry loop-integral parity constraints. The bridge has no continuous loop-power slot and is excluded from parity.

Momentum declarations serve two distinct roles. `loopExternalMomenta` specifies independent directions entering the loop, while `independentExternalMomenta` specifies the magnitudes of momenta that remain outside it. The package constructs the required Gram coordinates and checks whether these declarations are complete, overcomplete, or undercomplete. It never infers a role from a symbol's name.

Initialization returns the parsed topology, contact-reachable sectors, canonical packs, zero points, normalization data, and capability reports. A typical high-level sequence is

```
context = DSInit[caseInput];
seedData = DSSeeds[context];
allSeeds = DSAllSeeds[seedData];
generated = DSGenerateIBP[allSeeds,
  {a[v1],-1,3},{a[v2],-1,3},{b[1],-2,3},{b[2],-2,3}];
linearData = DSLinear[generated,context];
```

The same example also demonstrates custom coordinates through `DSRedefineParameters`, fixed-bridge magnitudes, sector contact maps, and exact round trips between user and internal scalar products.

3 From SK rules to integral families

3.1 Vertices and external phases

Each vertex is an association containing `id`, `vertexType`, and `externalLegEnergy`. The vertex type fixes the sign of the external phase, and the energy is the independent phase variable. The parser normalizes these signs once; all subsequent time derivatives and contact rules use the same convention.

3.2 Propagator types

The propagator classes are massive full, massless full, massive cross, and massless cross or fixed lines. Massive full lines have a two-theta contact system and a Hankel-type function space, while massless full lines use a two-vertex quotient representation. Cross lines have no ordinary two-theta shrink channel. Fixed tree lines carry external magnitudes but no continuous loop-integral power.

A standard line uses `id`, `endpoints`, `momentum`, and `massType`. Massive lines additionally require `nu`. A custom massive function system may be supplied through `functionSystem`; no physical type is inferred from a mass parameter alone.

3.3 Contact-reachable sectors

The root topology determines loop number, cycle/bridge roles, and propagator schemas. Contact operations only shrink lines, merge vertex representatives, and update zero points and normalization. They do not create a new unrelated family or silently reduce the structural loop number.

Massive contact terms are compiled from the Wronskian system. Simultaneous contacts merge a vertex only once, while integer shifts from all selected lines are accumulated. Uncut propagators are canonicalized in the target sector. Every reachable sector retains the metadata required by seeds, derivatives, and serializers.

4 Integral representations

In full mode the public integral head is

```
J[aList,linePacks,ispList]
```

The first slot contains compact vertex-time powers. Each full-line pack has the form $\{b_e, n_{e,1}, n_{e,2}\}$, where b_e is a continuous denominator power and the two entries are discrete building-block states. A shrunk line uses the corresponding compact bS representation. ISP powers are collected in the third slot and are measured from zero. Positive ISP powers are numerators; explicitly requested negative powers are also allowed.

In `ibpMode->"timeOnly"`, the public representation is instead

```
J[sectorKey,timeShifts,stateBits]
```

Here `sectorKey` is a fixed-width string of full/shrunk bits in root-line order. The other two slots contain compact time shifts and the sector-local discrete-state registry. The two schemas are intentionally distinct; no adapter silently rewrites full-loop Kira artifacts into the time-only representation.

5 Massive function systems, derivatives, and EOM

A massive building block is described by a 2×2 first-order system. Its derivative matrix and Wronskian determine, respectively, derivative shifts and contact-shrink shifts. These two data channels are not interchangeable.

After a generator acts, any Hankel state of second order or higher is immediately reduced by the equation of motion. This happens before linear-system construction, so no symbolic $n \geq 2$ state is passed to a backend. Contact shifts use the compiled Laurent coefficients and preserve the associated zero-point and prefactor changes.

Parameter derivatives act on both the integral and its explicit coefficients. The package also accounts for sector normalization, so a derivative of $J_s = N_s I_s$ includes the $D(\log N_s)$ term and, across a contact, the appropriate N_s/N_t ratio.

6 Massless quotients and time IBP

Massless full propagators retain an ordered two-vertex three-slot pack. Regular and delta terms are handled in that same representation, including contact shrink and coincident cancellation. Several full lines joining one vertex pair are treated by a common-theta construction rather than by independently duplicated contact rules.

An odd subset of lines in a shared bundle determines the contact transition. Its coefficient and target-sector mapping are evaluated once, and duplicate transitions are removed by source, target, and selected-line identity. Massless fixed lines do not acquire a fictitious loop parity constraint.

The direct pure-time/tree path uses the same physical contact data but presents them in a sector-tagged public representation. Formula-based tree reduction is available only for massive-only quotients whose derivation has been certified. In all other cases, the package returns a structured `PendingRederivation` result.

7 Topology, scalar products, ISP, and symmetry

The required topology input includes `vertices`, `lines`, `loopMomenta`, `loopExternalMomenta`, and `independentExternalMomenta`. ISP definitions are required when the propagators do not span all independent scalar products. Every ISP must be closed under the selected loop momentum generators.

The package distinguishes loop-external directions from loop-free magnitudes. Default loop coordinates are $\texttt{ssij} = \sqrt{\texttt{sp}[k_i, k_j]}$, with the derivative convention

$$\frac{\partial}{\partial \texttt{ssij}} = 2\,\texttt{ssij}\,\frac{\partial}{\partial \texttt{sp}[k_i, k_j]}. \tag{7.1}$$

Custom coordinates are introduced through `KinematicRules` or `DSRedefineParameters`. Undercomplete declarations stop initialization; overcomplete declarations may permit symbolic seed generation but disable unique derivatives and backend export.

Symmetry rules are applied as directed replacements. Users must choose one canonical representative for each equivalence class and provide oriented rules only. The symmetry of `sp` itself does not imply a graph symmetry: masses, energies, and momentum routing must also match.

8 IBP generators and atomic operations

Time generators differentiate with respect to conformal times. Momentum generators include loop scalings and the independent external directions required by the declared loop basis. The complete set is constructed before any index sampling.

The public atomic operations include `dtau`, `dqq`, `dqk`, and `ds`. The first three generate time and momentum IBP relations, while `ds` is the total parameter derivative used by the differential-equation workflow. Conversion helpers `rep2innerform` and `rep2outform` translate between user and internal coordinates without changing integral indices. `rep2Integrand` exposes an inert integrand for inspection; it does not integrate or generate a relation.

9 Seeds and continuous-index envelopes

9.1 Symbolic templates and shift metadata

`DSSeeds` enumerates the independent discrete templates before continuous sampling. Massive vertices use the four 0/1 states; massless quotients use their proved canonical representatives. EOM, symmetry, and sector canonicalization are applied while continuous indices remain symbolic.

`DSAllSeeds` returns the flat template list, while `DSSeedGroups` and `DSSeedGroupMetadata` expose the recommended grouping. `DSMetaSeedRange` records the affine shift set of every continuous index. A flat input is treated as one group; each top-level element of a nested input is treated as a separate group.

A uniform target envelope is written as

```
ibp = DSGenerateIBP[allSeeds,{L,U}];
```

A detailed envelope assigns an interval to every root index. For an index with shift set Δ, the corresponding seed interval is the intersection of all inverse images:

$$x \in [L - \min \Delta,\ U - \max \Delta]. \tag{9.1}$$

For ISP indices, the lower endpoint is also bounded below by the user target lower endpoint. Empty inverse ranges are reported explicitly and disable the relevant completeness claim.

9.2 Seed-envelope cutting

`SeedEnvelopeCut` filters the inferred seed points before parity filtering and template substitution. The following calls disable cutting, use the default, and provide custom parameters:

```
ibp0 = DSGenerateIBP[allSeeds,{-1,1},SeedEnvelopeCut->None];
ibp1 = DSGenerateIBP[allSeeds,{-1,1},SeedEnvelopeCut->Automatic];
ibp2 = DSGenerateIBP[allSeeds,{-1,1},SeedEnvelopeCut->{10,4,5,4}];
```

Let C_+ be the sum of positive continuous slots, C_- the absolute sum of negative slots, and $T = C_+ + C_-$. For $C_+ \geq C_-$, acceptance requires $T \leq Ex_1$ and $C_- \leq Ex_2$; otherwise it requires $T \leq Ex_3$ and $C_+ \leq Ex_4$. The default for a group of N continuous slots is $\{N + 3, 4, 4, 4\}$. Every item of an explicit four-tuple may be an integer or `Automatic`, in which case only that default item is substituted.

The returned `seedEnvelopeCut` metadata records the resolved parameters, accepted and skipped point counts, and groups emptied by the cut. `None` restores the uncut point domain.

9.3 Canonical-support cutting

`TotalPowerLimit` is an independent filter on canonical equation support:

```
ibp3 = DSGenerateIBP[allSeeds,{-1,1},TotalPowerLimit->7];
ibp4 = DSGenerateIBP[allSeeds,{-1,1},
  SeedEnvelopeCut->{10,4,5,4},TotalPowerLimit->7];
```

For an integer m, the package first combines equal integrals and removes zero coefficients. It then retains an equation only if every canonical integral in its actual support lies in the per-slot target box and has continuous-index absolute sum at most m. The default `None` disables this filter. The two cut options can be used independently or together.

The full-mode fast path decomposes Association-based sealed templates into integral skeletons, integer shifts, and coefficient pieces, then memoizes canonical results by raw integral. Raw expression lists and failed templates use the original symbolic path. This changes performance, not the meaning of either cut option.

These options do not prove closure for an arbitrary target or basis. `completeSystemQ` describes generation completeness within the selected envelope and cutting policy. A new target, basis, or cut setting still requires an external reduction check that every requested target reduces to the requested masters.

10 Linear data and the Kira workflow

`DSLinear` converts generated relations into backend-neutral `linearData`. Its `integralList` is the authoritative backend order, and equations are deduplicated by their mathematical coefficients, constant terms, and nonlinear terms rather than by their provenance. `DSReorderIntegrals` explicitly changes that order before basis or backend construction. `DSUserMI` attaches a reversible ordered basis of single integrals or homogeneous linear combinations and constructs analytic derivative targets.

A formal Kira workflow has two stages: optional pre-reduction for basis exploration and a formal stage using the active basis and its minimal derivative closure. `DSKiraPlan` constructs the plan, and `DSKiraExport` serializes equations, maps, jobs, and manifest information without running Kira.

For a massive bubble, `numericStage->"postDerivative"` substitutes the declared parameter rules only after the analytic derivatives have been constructed; it does not necessarily make the reduction purely numerical. In the bundled example, `dim`, `nu`, `etaNu`, and `analyticRegulator` take exact rational values, but the kinematic variables remain symbolic during reconstruction. Kira therefore reconstructs in `dsc2` and `ip0`, and the resulting DE matrices remain symbolic in `ss11` and P_0. Comparison with the analytic reference at a fixed numerical point is a separate final check.

After external reduction, `DSKiraImport` checks manifest identity, target coverage, master order, and RHS closure. `DSDE` constructs matrices and sources in the active basis, and `DSScaleCheck` checks the applicable symbolic, fixed-section, or exact-point scaling certificate. A global residual master outside the requested target closure is not by itself a failure; acceptance is based on the requested targets and their RHS.

11 Initialization, messages, and inspection

`DSInit` validates topology, ISP closure, kinematics, and sector metadata. `DSInfo` returns initialization summaries. `DSKinematics` and `DSParameterNotation` expose coordinate

proposals and current notation, while `DSRedefineParameters` performs a complete reinitialization under new rules.

`DSMessagesOn`, `DSMessagesOff`, and `DSMessagesQ` control optional information and warnings. Fatal errors remain visible. `AuditLevel` controls whether cached producer certificates are reused or full development audits are recomputed. `DSPublicAPI` returns the public function groups and the high-level options actually present in the loaded package.

12 Pure massive bubble closed loop

The bundled two-vertex massive bubble uses the $--$ branch, even parity on both cycle propagators, and equal vertex energies. Its target envelope is $a_v \in [0, 3]$ and $b_e \in [-3, 5]$, covering the 19-element basis and all derivative support. The example uses `SeedEnvelopeCut->Automatic` and `TotalPowerLimit->None`.

A fresh run generates 20228 IBP relations before linearization, followed by 2442 canonical relations, 2460 formal equations, and 1317 source integrals. Kira reduces all 300 requested targets, leaving none unreduced. The active basis has 19 masters; five additional global masters do not appear on the right-hand sides of the requested reductions. Import, 19×19 differential equations, and scaling pass 23 checks. The physical P_0, backend `ip0`, and physical k_s matrices each agree with the frozen analytic reference in all 361 entries.

The example requires an external workspace selected by `DSIBP_KIRA_WORKSPACE`. The package writes inputs but does not start Kira. After Kira finishes, `post_kira_check.wl` reads the existing results without regenerating inputs. Repository files retain only lightweight input and result summaries, not databases, reduction tables, caches, or runtime logs.

13 Tree vertex families

The direct tree workflow exposes `DSTreeSeeds`, `repIterative`, `DSTreeNaiveIBP`, `DSTreeNaiveDE`, and `DSTreeDLogDE`. It constructs sector-tagged relations, ordered masters, normalized differential equations, and scaling checks from the same contact and normalization metadata.

`DSTreeNaiveIBP` solves one-step raising relations in a specified basis. `DSTreeNaiveDE` uses that reduction to construct the naive connection, while `DSTreeDLogDE` constructs the block-structured dlog connection. For massless full quotients, formula-based reduction is available only after the required identities have been rederived.

14 Common errors and current boundaries

Malformed topology, missing momentum directions, incomplete ISP closure, unknown or repeated range indices, symbolic discrete indices, and invalid cut specifications are reported as structured failures. Empty seed ranges are not silently ignored. Backend export also rejects unresolved internal coordinates, unsupported complex conventions, stale manifests, and incomplete reduction artifacts.

dSIBP does not run reduction internally. It does not infer physical symmetry from symbol names, and it does not turn a finite-envelope completeness claim into a proof for arbitrary targets. Formula-based tree reduction remains limited to the certified massive-only quotient cases. Reference summaries record their evidence baseline and are not silently reused as proof for a different generation policy.

15 Public API summary

Function group	Purpose
`DSInit`, `DSInfo`, `DSKinematics`, `DSParameterNotation`, `DSRedefineParameters`	Initialization, topology and coordinate inspection, and redefinition.
`DSSeeds`, `DSAllSeeds`, `DSSeedGroups`, `DSSeedGroupMetadata`, `DSMetaSeedRange`	Symbolic discrete templates, grouping, and shift metadata.
`DSGenerateIBP`	Continuous expansion with `SeedEnvelopeCut` and `TotalPowerLimit`.
`DSLinear`, `DSReorderIntegrals`, `DSUserMI`	Backend-neutral equations, authoritative order, and active bases.
`DSKiraPlan`, `DSKiraExport`, `DSKiraImport`	Kira planning, serialization, and verified result import.
`DSDE`, `DSScaleCheck`	Differential equations and scaling certificates.
`DSTreeSeeds`, `repIterative`, `DSTreeNaiveIBP`, `DSTreeNaiveDE`, `DSTreeDLogDE`	Direct pure-time/tree generation and reduction.
`dtau`, `dqq`, `dqk`, `ds`	Atomic time, momentum, and parameter derivatives.
`rep2innerform`, `rep2outform`, `rep2Integrand`, `symmetry`, `repSymmetry0`	Coordinate conversion, inert integrands, and symmetry access.
`DSMessagesOn`, `DSMessagesOff`, `DSMessagesQ`, `DSPublicAPI`	Message control and public-interface inspection.

The exact options of a function are those returned by `Options[function]`. `DSGenerateIBP` exposes `SeedEnvelopeCut` (default `Automatic`) and `TotalPowerLimit` (default `None`), together with the shared audit and progress options.